

HashiCorp Vault

Utilisation & Intégration

Formation 2 jours — 14 heures
Développeurs, DevOps, SRE

Ihab ABADI — Utopios

Programme — Jour 1 : consommer Vault

1. Consommer Vault

concepts, CLI, API REST, KV v2

2. Authentification applicative

AppRole, secret zéro, response wrapping

3. Politiques et moindre privilège

HCL, entities, politiques templatées

Programme — Jour 2 : intégrer Vault

4. Secrets dynamiques et transit

PostgreSQL éphémère, chiffrement as a service

5. Vault dans Kubernetes

Agent, Injector, Vault Secrets Operator, CSI

6. CI/CD et bonnes pratiques

JWT/OIDC, bound claims, synthèse

Fil rouge : **PayLink**, fintech fictive — et **MediTrack**, son service santé.

Le fil conducteur de la formation

Trois règles, rappelées à chaque module :

- **Jamais de secret en clair**
ni dans le code, ni dans la config, ni dans les variables de CI
- **Une identité par service**
le pipeline de PayLink n'est ni celui de MediTrack, ni « la CI »
- **Le moindre privilège par défaut**
on part de zéro droit et on ajoute, jamais l'inverse

Périmètre et lab

Ce que cette formation couvre : **consommer** Vault et l'**intégrer** à vos applications.

Hors périmètre (survolé pour comprendre le modèle) : installation d'un cluster, unseal de production, réplication, sauvegardes — le métier de l'administrateur Vault.

Le lab : **Podman** (Vault 1.20 en mode dev, PostgreSQL 16) et un cluster **kind** pour la partie Kubernetes.

Toutes les commandes ont été rejouées ; les sorties affichées sont réelles.

Jour 1

Consommer Vault

Module 1

Vault côté consommateur : concepts, CLI & API REST

HashiCorp Vault — Utilisation & Intégration
Jour 1 — Durée : 2 h

Objectifs du module

À la fin de ce module, vous serez capable de :

- Expliquer le problème que Vault résout (sprawl, rotation, audit)
- Situer les briques vues du consommateur : mounts, paths, tokens, leases, policies
- Lire, écrire et versionner des secrets avec la CLI `vault` (KV v2)
- Consommer l'API REST de Vault avec `curl` et interpréter ses codes de retour
- Créer un token restreint et vérifier ses capabilities

Plan

1. Le problème des secrets
2. Architecture vue du consommateur
3. La CLI `vault` : le moteur KV v2
4. L'API REST
5. Tokens et policies : premier contact
6. Récap et quiz

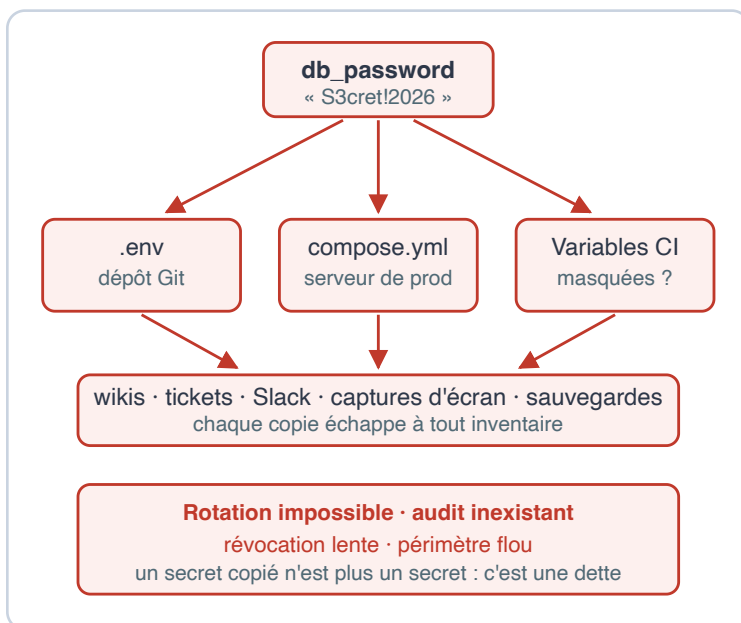
Fil rouge : **PayLink**, notre API de paiement, doit cesser de stocker ses credentials en clair.

1. Le problème des secrets

Le sprawl : des secrets partout

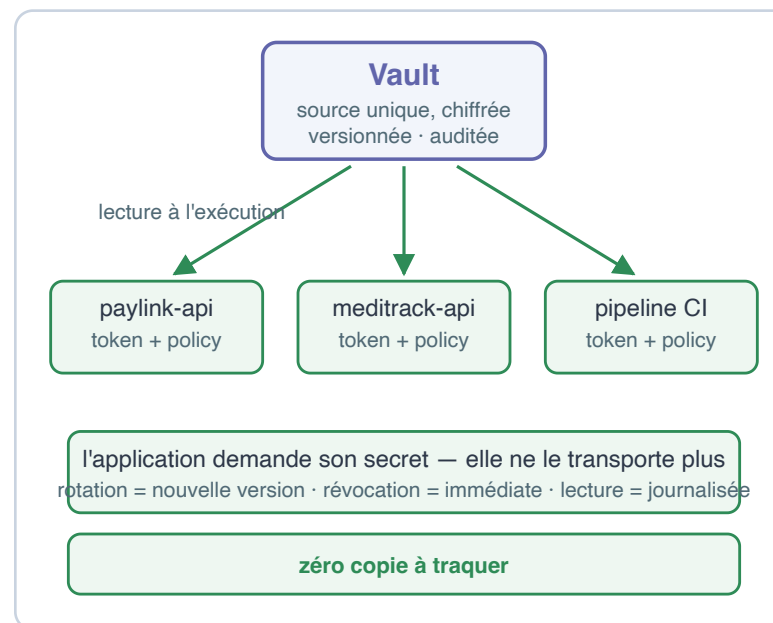
Avant — le secret est copié

db_password vit partout, personne ne sait combien de copies existent



Avec Vault — le secret est demandé

une source unique, un accès authentifié, une trace



Un secret copié n'est plus un secret : c'est une **dette**.

Ce que ce sprawl coûte

- **Rotation impossible** : changer `db_password` = retrouver toutes ses copies, redéployer, croiser les doigts
- **Audit inexistant** : qui a lu ce mot de passe ? Quand ? Aucune trace
- **Révocation lente** : un départ, une fuite, un laptop volé
→ des jours pour tourner les credentials
- **Périmètre flou** : le mot de passe de prod traîne dans un `docker-compose.yml` de dev

Les fuites viennent rarement d'une cryptanalyse brillante : elles viennent d'un secret **posé au mauvais endroit**.

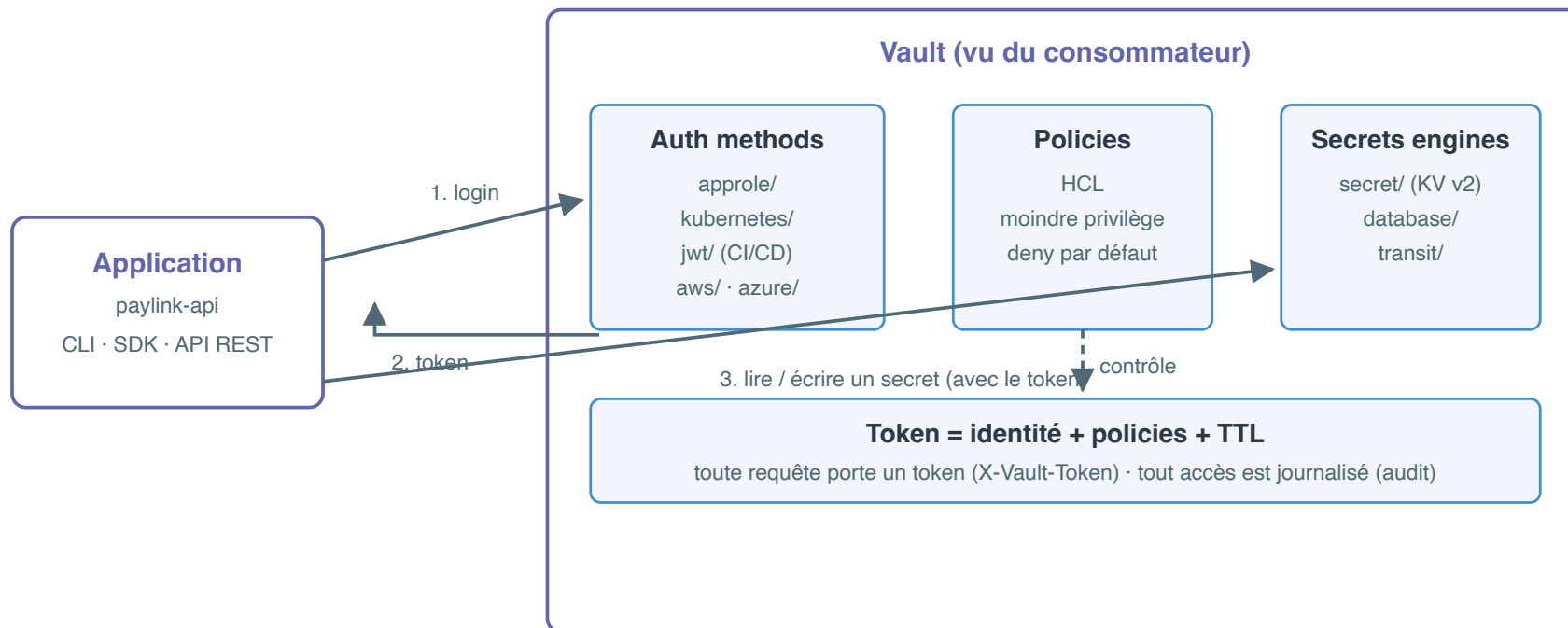
Ce que Vault change

Avant	Avec Vault
Secrets copiés dans N fichiers	Une source unique , chiffrée au repos
Accès implicite (qui a le fichier)	Accès authentifié et autorisé (token + policy)
Aucune trace	Journal d'audit de chaque lecture/écriture
Rotation = projet d'un mois	Nouvelle version du secret, les clients relisent
Credentials éternels	Leases : durée de vie, renouvellement, révocation

Le principe : l'application **demande** son secret à l'exécution, elle ne le **transporte** plus.

2. Architecture vue du consommateur

Vault vu par une application



Trois portes d'entrée (UI, CLI, API), **une seule API HTTP** derrière.

Mounts et paths : le système de fichiers des secrets

```
http://vault:8200/v1/  
|  
+-- secret/          <- mount KV v2 (nos secrets statiques)  
|   +-- paylink/  
|       +-- api      { db_user, db_password, api_key }  
|       +-- smtp     { host, user, password }  
|   +-- meditrack/  
|       +-- api      (isolé de paylink par policy)  
|  
+-- sys/             <- mount système (health, policies...)  
+-- cubbyhole/       <- stockage privé par token  
+-- identity/        <- entités et alias
```

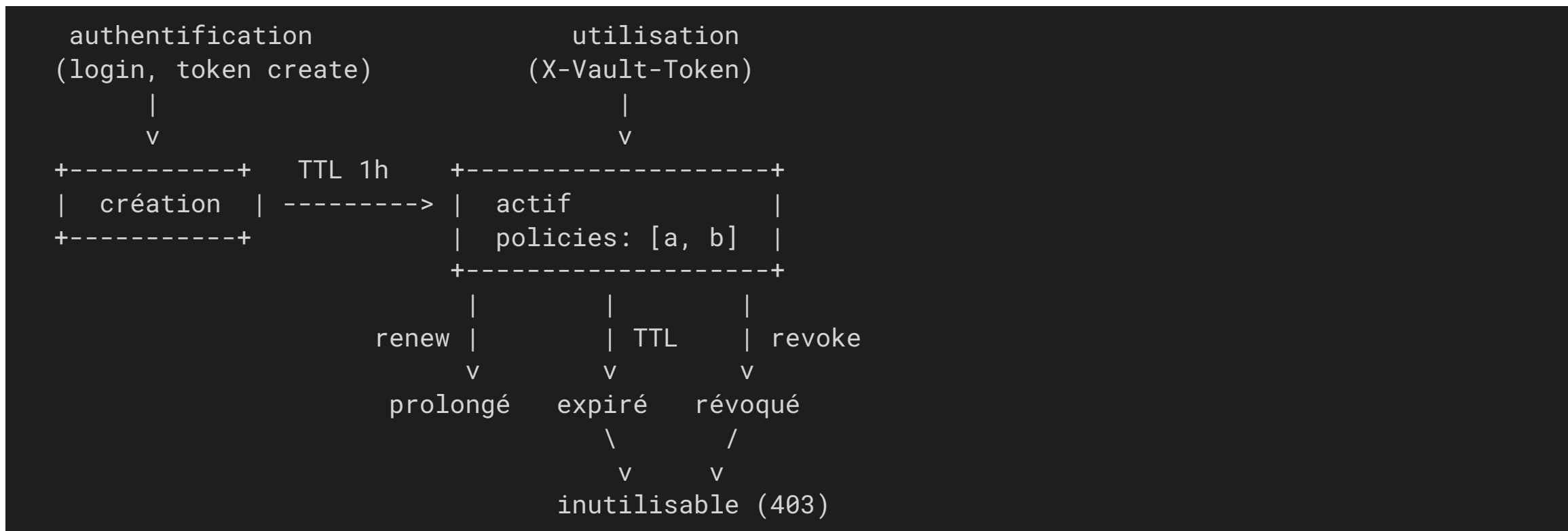
- Un **mount** = un moteur de secrets branché sur un préfixe
- Un **path** = l'adresse d'un secret sous ce mount
- Tout est adressé par chemin : la policy s'applique **au chemin**

Le token : votre identité auprès de Vault

- **Toute** requête (sauf `sys/health`) est authentifiée par un token
- En HTTP : en-tête `X-Vault-Token: <token>` – en CLI : variable `VAULT_TOKEN` (ou `vault login`)
- Un token porte des **policies** (ce qu'il peut faire) et un **TTL** (combien de temps)

Le token racine du lab (`root-token-formation`) est un outil de formation : **en production, personne ne travaille avec un token root.**

Le cycle de vie d'un token



Un token n'est pas un mot de passe : c'est un **bail temporaire**.

Leases et policies : le survol utile

Lease (bail)

- Tout secret *dynamique* (module 4) et tout token vivent sous un lease
- `lease_duration` dans les réponses : durée avant expiration
- Renouvelable (`renew`), révocable (`revoke`) — l'expiration nettoie seule

Policy (module 3 en détail)

- Document HCL : des chemins + des capabilities (`read`, `create`, `list` ...)
- **Déni par défaut** : ce qui n'est pas explicitement accordé est refusé
- Attachées aux tokens ; la policy `default` est presque vide

Ce que nous ne couvrons pas

Cette formation vous apprend à **consommer** Vault, pas à l'opérer.

Hors périmètre (survolé uniquement pour comprendre le modèle) :

- Installation et dimensionnement d'un cluster de production
- **Unseal** de production (auto-unseal KMS, clés de Shamir)
- Clustering, réplication, disaster recovery
- Sauvegardes et mises à jour du serveur

Notre lab tourne en **mode dev** : démarré dé-scélé, stockage en mémoire, token root connu. Parfait pour apprendre, interdit en prod.

Le lab de la formation

Un conteneur Podman `vault` (image `docker.io/hashicorp/vault:1.20`, Vault 1.20.4), réseau `vault-lab`, port 8200 publié, token racine `root-token-formation`.

```
cd code/lab && ./lab-up.sh
```

Alias recommandé pour la CLI (la CLI est dans le conteneur) :

```
alias v='podman exec -e VAULT_ADDR=http://127.0.0.1:8200 \  
-e VAULT_TOKEN=root-token-formation vault vault'
```

Trois accès :

- CLI : `v status`
- API : `curl http://127.0.0.1:8200/v1/sys/health`
- UI : `http://127.0.0.1:8200` (token : `root-token-formation`)

3. La CLI `vault` : le moteur KV v2

Écrire et lire : `kv put` / `kv get`

```
v kv put secret/paylink/api \  
  db_user=paylink_app db_password='S3cret!2026' api_key=pk_live_51J
```

```
===== Secret Path =====
```

```
secret/data/paylink/api
```

```
===== Metadata =====
```

Key	Value
-----	-------

---	-----
-----	-------

created_time	2026-07-07T19:05:19.126661792Z
--------------	--------------------------------

version	1
---------	---

- `kv get secret/paylink/api` relit données **et** métadonnées
- Remarquez le chemin renvoyé : `secret/data/paylink/api` — on y revient

Chemin logique vs chemin réel

Ce que vous tapez (CLI)

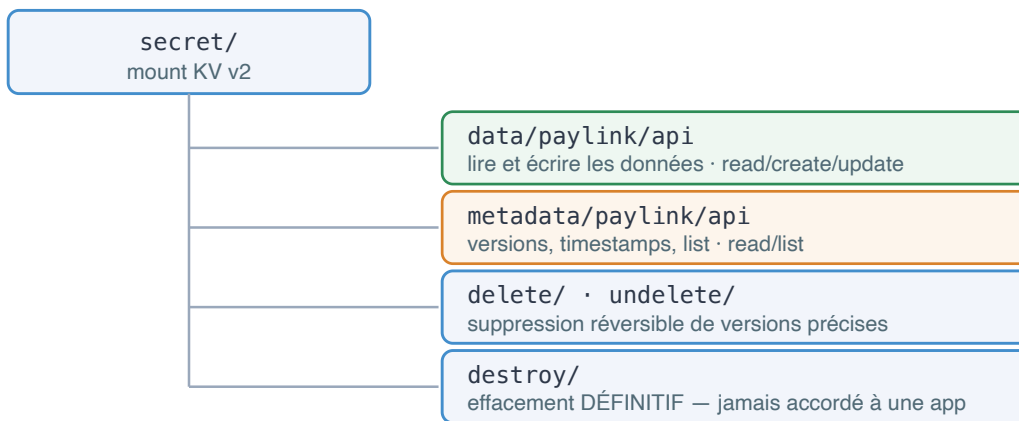
`vault kv get secret/paylink/api`
chemin logique — la CLI complète
le chemin pour vous

insère
data/

Ce que l'API reçoit (KV v2)

`GET /v1/secret/data/paylink/api`
chemin réel — c'est LUI que la policy doit matcher
l'oublier dans une policy KV v2 = 403 garanti

L'arbre complet d'un secret KV v2 — une famille d'opérations par préfixe



Conséquence : le moindre privilège

On accorde la lecture des données
sans le droit de lister.

On accorde l'écriture
sans la suppression.

Chaque famille d'opérations ayant
son préfixe, la policy découpe au verbe près.

En **curl** et en **policy**, c'est à vous d'écrire **data/**.

KV v2 : chaque écriture crée une version

```
v kv put secret/paylink/api \  
    db_user=paylink_app db_password='Rot4ted!2026' api_key=pk_live_51J
```

```
created_time    2026-07-07T19:05:19.320325124Z  
version         2
```

La rotation d'un mot de passe devient banale : on écrit, l'ancienne valeur reste consultable.

```
v kv get -version=1 secret/paylink/api    # relit la version 1
```

Attention : **kv put** remplace toutes les clés du secret.

kv patch : fusionner au lieu de remplacer

patch fusionne : seules les clés fournies changent, les autres sont reprises — mais le résultat est quand même une **nouvelle version**.

```
v kv patch secret/paylink/api api_key=pk_live_99Z
```

```
created_time      2026-07-07T19:05:19.520495063Z  
version           3
```

Exploiter les sorties en script

```
v kv get -field=db_password secret/paylink/api
```

```
Rot4ted!2026
```

```
v kv get -format=json secret/paylink/api | jq -r .data.data.api_key
```

`-field` : une valeur brute ; `-format=json` : la réponse API complète (vos valeurs sous `data.data`).

Métadonnées et historique des versions

```
v kv metadata get secret/paylink/api
```

```
===== Metadata Path =====
```

```
secret/metadata/paylink/api
```

Key	Value
---	----
created_time	2026-07-07T19:05:19.126661792Z
current_version	3
oldest_version	0
custom_metadata	<nil>

Suivi de chaque version : `created_time`, `deletion_time`, `destroyed`.

`custom_metadata` : vos étiquettes (propriétaire, criticité...) —
jamais de secret dedans.

Supprimer et restaurer

```
v kv delete secret/paylink/api # soft delete (version courante)
```

```
Success! Data deleted (if it existed) at: secret/data/paylink/api
```

```
v kv undelete -versions=3 secret/paylink/api # la version revient  
v kv destroy -versions=1 secret/paylink/api # destruction DEFINITIVE
```

`delete` pose un `deletion_time` (récupérable) ; `destroy` efface la donnée de la version, irréversible ; `metadata delete` supprime tout l'historique.

Explorer : `kv list`

```
v kv list secret/paylink
```

```
Keys
```

```
----
```

```
api
```

```
smtp
```

`list` montre les **clés** sous un préfixe, jamais les valeurs (capability `list` sur `secret/metadata/...`).

Convention PayLink : `secret/<service>/<usage>`.



Démo 1.1

CLI et KV v2 : le cycle de vie complet d'un secret PayLink

`demos/module-01/demo-1-1-cli-kv.md` — 15 min

4. L'API REST

Anatomie d'une requête Vault

```
GET http://127.0.0.1:8200/v1/secret/data/paylink/api
      \_____/ \_/ \_____/
        serveur  API  mount + chemin réel
                (toujours v1)
```

En-tête : X-Vault-Token: <votre token>

- **Tout** passe par HTTP : la CLI et l'UI ne font qu'appeler cette API
- Préfixe fixe `/v1/` (version de l'API, pas du secret)
- Lecture = `GET`, écriture = `POST` (ou `PUT`), suppression = `DELETE`

Le seul endpoint sans token : le healthcheck

```
curl -s http://127.0.0.1:8200/v1/sys/health | jq
```

```
{  
  "initialized": true,  
  "sealed": false,  
  "standby": false,  
  "version": "1.20.4"  
}
```

Code 200 = actif ; 503 = scellé ; 501 = non initialisé.

Lire un secret en curl

```
curl -s -H "X-Vault-Token: root-token-formation" \  
http://127.0.0.1:8200/v1/secret/data/paylink/api | jq
```

```
{  
  "request_id": "a1806ca7-fd40-7ef3-8e94-a9cfae870139",  
  "data": {  
    "data": {  
      "api_key": "pk_live_99Z",  
      "db_password": "Rot4ted!2026",  
      "db_user": "paylink_app"  
    },  
    "metadata": { "version": 3 }  
  },  
  "mount_type": "kv"  
}
```

La réponse porte aussi `lease_id`, `renewable`, `auth` — slide suivante.

L'enveloppe de réponse KV v2

```
{  
  "request_id": ...           <- identifiant de la requête (audit)  
  "lease_id", "lease_duration", "renewable"  
                                <- vides ici : un secret KV n'a pas de lease  
  "data": {  
    "data": { ... }           <- VOS paires clé/valeur  
    "metadata": { ... }       <- version, created_time, deletion_time  
  }  
}
```

- En KV v2, vos valeurs sont sous **data.data** – deux niveaux
- Les métadonnées sont sous **data.metadata**
- C'est ce double niveau qui explique le **.Data.data.xxx** des templates (module 5)

Écrire un secret et cibler une version

```
curl -s -H "X-Vault-Token: root-token-formation" \
  -X POST -d '{"data":{"url":"https://hooks.paylink.io/pay","hmac_secret":"whk_7f3a"}}' \
  http://127.0.0.1:8200/v1/secret/data/paylink/webhook | jq .data
```

```
{
  "created_time": "2026-07-07T19:06:34.52339549Z",
  "version": 1
}
```

- Le corps doit envelopper vos clés dans `{"data": ...}` (KV v2)
- Lecture d'une version précise : `?version=2` en query string

```
curl -s -H "X-Vault-Token: root-token-formation" \
  "http://127.0.0.1:8200/v1/secret/data/paylink/api?version=2" | jq .data
```

Les codes de retour à connaître

Code	Signification	Réflexe
200	Succès avec corps JSON	lire <code>data</code>
204	Succès sans corps (delete...)	rien à parser
400	Requête invalide (JSON, paramètres)	relire le corps envoyé
403	<code>permission denied</code> : token absent, invalide ou policy insuffisante	<code>token capabilities</code>
404	Chemin inexistant ou secret supprimé	vérifier mount + <code>data/</code>
503	Vault scellé ou en maintenance	<code>sys/health</code>

```
{"errors":["2 errors occurred:\n\t* permission denied\n\t* invalid token\n\n"]}
```

Un 403 sur un chemin qui existe et un 404 se ressemblent volontairement :
Vault ne confirme jamais l'existence d'un chemin à un token non autorisé.



Démo 1.2

L'API REST en direct : curl, versions, et un 403 instructif

`demos/module-01/demo-1-2-api-rest.md` — 10 min

5. Tokens et policies : premier contact

Créer un token restreint

Policy `paylink-read` (lecture seule sur les secrets PayLink) :

```
path "secret/data/paylink/*" {  
  capabilities = ["read"]  
}  
path "secret/metadata/paylink/*" {  
  capabilities = ["read", "list"]  
}
```

```
v policy write paylink-read /tmp/paylink-read.hcl  
v token create -policy=paylink-read -ttl=1h -format=json
```

```
client_token: hvs.CAESIGw5Fof2...  
policies: ['default', 'paylink-read']  
lease_duration: 3600
```

Inspecter et vérifier : lookup, capabilities

```
v token lookup hvs.CAESIGw5Fof2...
```

```
creation_ttl      1h
policies          [default paylink-read]
ttl              59m59s
```

capabilities répond à la question « ce token peut-il ? » **avant** de subir un 403 :

```
v token capabilities hvs.CAESIGw5Fof2... secret/data/paylink/api
read
v token capabilities hvs.CAESIGw5Fof2... secret/data/meditrack/api
deny
```

deny sans policy explicite : le **déni par défaut** en action.

Le 403 que vous vouliez voir

Avec le token restreint, la lecture passe... l'écriture non :

```
Error writing data to secret/data/paylink/api: Error making API request.
```

```
URL: PUT http://127.0.0.1:8200/v1/secret/data/paylink/api
```

```
Code: 403. Errors:
```

- La policy accorde `read`, pas `create` / `update` → 403
- C'est le comportement **souhaité** : PayLink lit ses secrets, seule la chaîne de provisionnement les écrit
- Le module 3 fera de vous des auteurs de policies



Exercice 1.1

**Le service Ledger entre dans Vault : lab, CLI, curl,
cycle de vie**

`exercices/module-01/exercice-1-1-premiers-secrets.md` — 30 min

Récap du module

- Le sprawl des secrets se règle par une **source unique**, auditée, à laquelle l'application **demande** ses secrets à l'exécution
- Vue consommateur : **mounts** → **paths**, accès par **token** (`X-Vault-Token`) porteur de **polices**, durées gérées par **leases**
- KV v2 : `put` remplace, `patch` fusionne, chaque écriture **versionne** ; `delete` est réversible, `destroy` non
- Chemin logique CLI ≠ chemin réel API : `data/` et `metadata/`
- API REST : enveloppe `data.data`, corps `{"data": ...}`, codes 200/403/404
- `token create -policy`, `lookup`, `capabilities` : vérifier avant de subir

Cheatsheet : `cheatsheet/module-01.md`

Quiz – questions 1 et 2

Question 1

Quel en-tête HTTP transporte votre identité dans chaque appel à l'API Vault ?

Question 2

Vous tapez `vault kv put secret/paylink/api ...`. Quel chemin l'API reçoit-elle réellement, et pourquoi ?

Quiz — questions 3 et 4

Question 3

Quelle est la différence entre `vault kv put` et `vault kv patch` sur un secret existant ?

Question 4

Après un `vault kv delete`, les données de la version sont-elles définitivement perdues ? Quelle commande change tout ?

Quiz – questions 5 et 6

Question 5

Un `curl` vers `/v1/secret/data/paylink/api` renvoie un code 403.
Citez deux causes possibles.

Question 6

En script shell, comment récupérer uniquement la valeur de `db_password` avec la CLI ? Donnez deux approches.

Quiz – questions 7 et 8

Question 7

Quel endpoint interrogez-vous pour vérifier l'état du serveur Vault sans présenter de token ?

Question 8

Dans la réponse JSON d'une lecture KV v2, à quel emplacement exact se trouvent vos paires clé/valeur ?

Quiz — questions 9 et 10

Question 9

`vault token capabilities <token> secret/data/meditrack/api` renvoie `deny` alors qu'aucune policy ne mentionne ce chemin. Est-ce un bug ?

Question 10

Qu'est-ce qu'un lease, et pourquoi est-ce une bonne nouvelle pour la sécurité de vos credentials ?

Questions ?

Module suivant : **Authentication applicative**
AppRole, secret zéro, response wrapping

Module 2

Authentication applicative

HashiCorp Vault — Utilisation & Intégration
Jour 1 — 2 h 30

Objectifs du module

À la fin de ce module, vous serez capable de :

- Expliquer pourquoi un token statique dans une application est un anti-pattern
- Configurer un rôle AppRole et obtenir un token applicatif en CLI et en API REST
- Livrer un secret_id par response wrapping et détecter sa compromission
- Situer les auth methods Kubernetes et cloud, et choisir selon votre contexte
- Entretenir un token applicatif (renew) et anticiper le re-login

Plan

1. Le problème du token statique
2. AppRole en profondeur
3. Le secret zéro et le response wrapping
4. Auth Kubernetes et auth cloud : principes
5. Vivre avec son token : renew et re-login

Pratique : démo 2.1, exercice 2.1, démo 2.2, TP 2, quiz.

1. Le problème du token statique

PayLink, vendredi, 17 h 58

Au module 1, nous lisons les secrets PayLink avec un token. Première idée de l'équipe pour l'API en production :

```
# .env de paylink-api (commité "temporairement" sur la branche)
VAULT_ADDR=https://vault.paylink.internal
VAULT_TOKEN=hvs.CAESILKWZtih3xx8pR0gDH4f... # créé à la main il y a 7 mois
```

Trois questions pour la salle :

- Qui d'autre a ce token ? (dépôt Git, CI, postes, sauvegardes, ex-collègues...)
- Que se passe-t-il si on le révoque pour le changer ?
- Dans l'audit log Vault, qui est « ce token » ?

Trois défauts rédhibitoires

Défaut	Conséquence concrète
Il fuit	Copié dans Git, la CI, les logs, les tickets, les screenshots. Une fuite = accès total, silencieux, depuis n'importe où
Il ne tourne pas	Le révoquer casse la prod ; personne n'ose. Il devient éternel – TTL infini de fait, contraire à tout référentiel de sécurité
Il n'a pas d'identité	<code>display_name: token</code> . Impossible de dire <i>quel service</i> a lu quoi, de scoper finement, de révoquer <i>un</i> consommateur

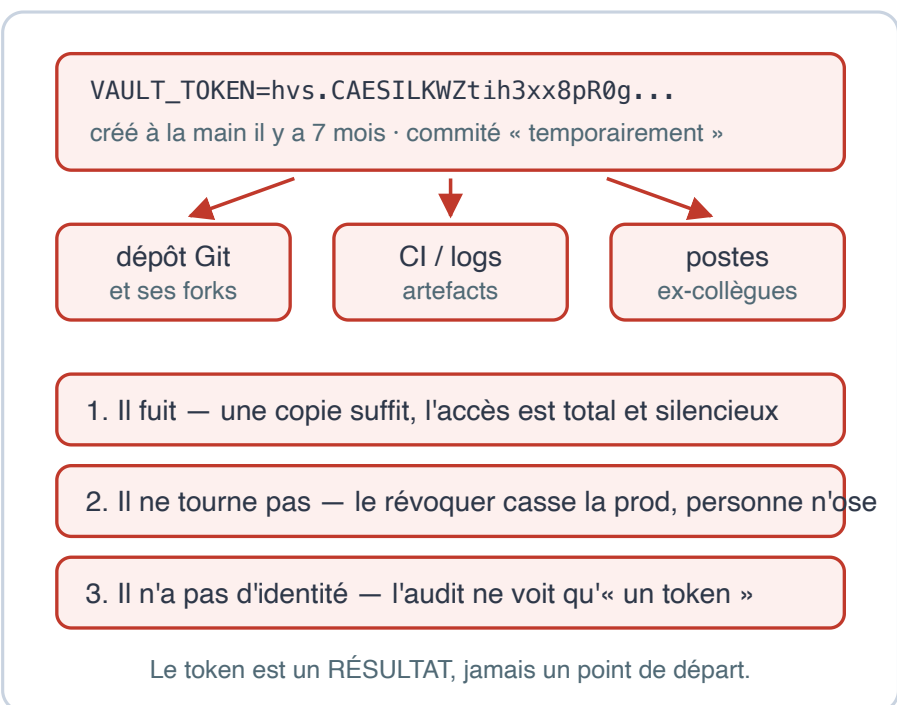
Le token n'est pas le problème – c'est le **token distribué à la main et durable** qui l'est.

Principe Vault : le token est un *résultat*, jamais un *point de départ*.
L'application prouve son identité, Vault lui délivre un token **court, scopé, renouvelable, traçable**.

Le renversement : de la distribution au login

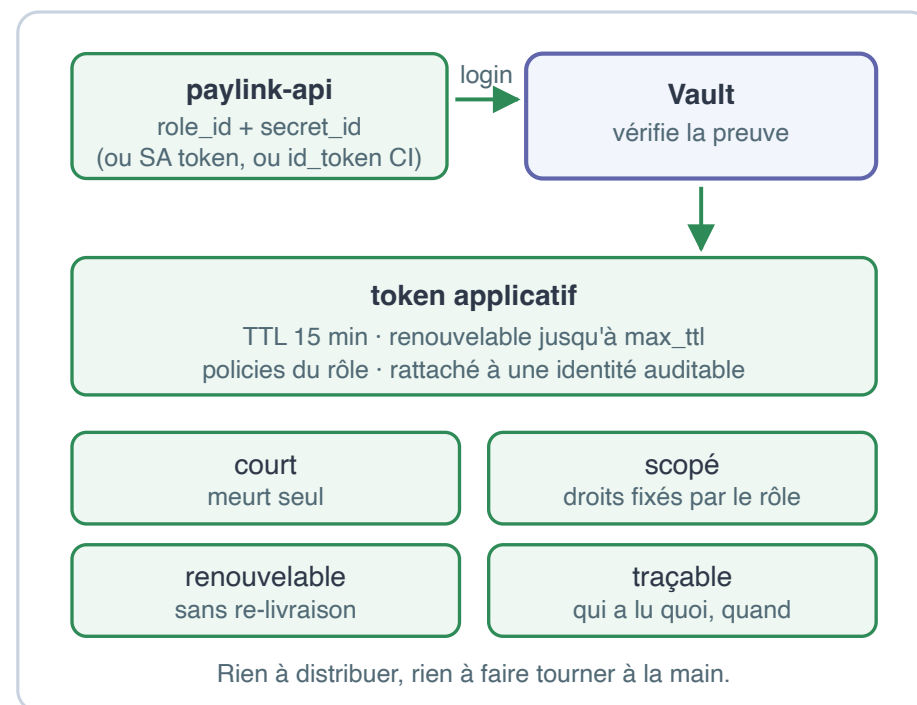
✗ Le token statique

distribué à la main, jamais tourné — un mot de passe en dur déguisé



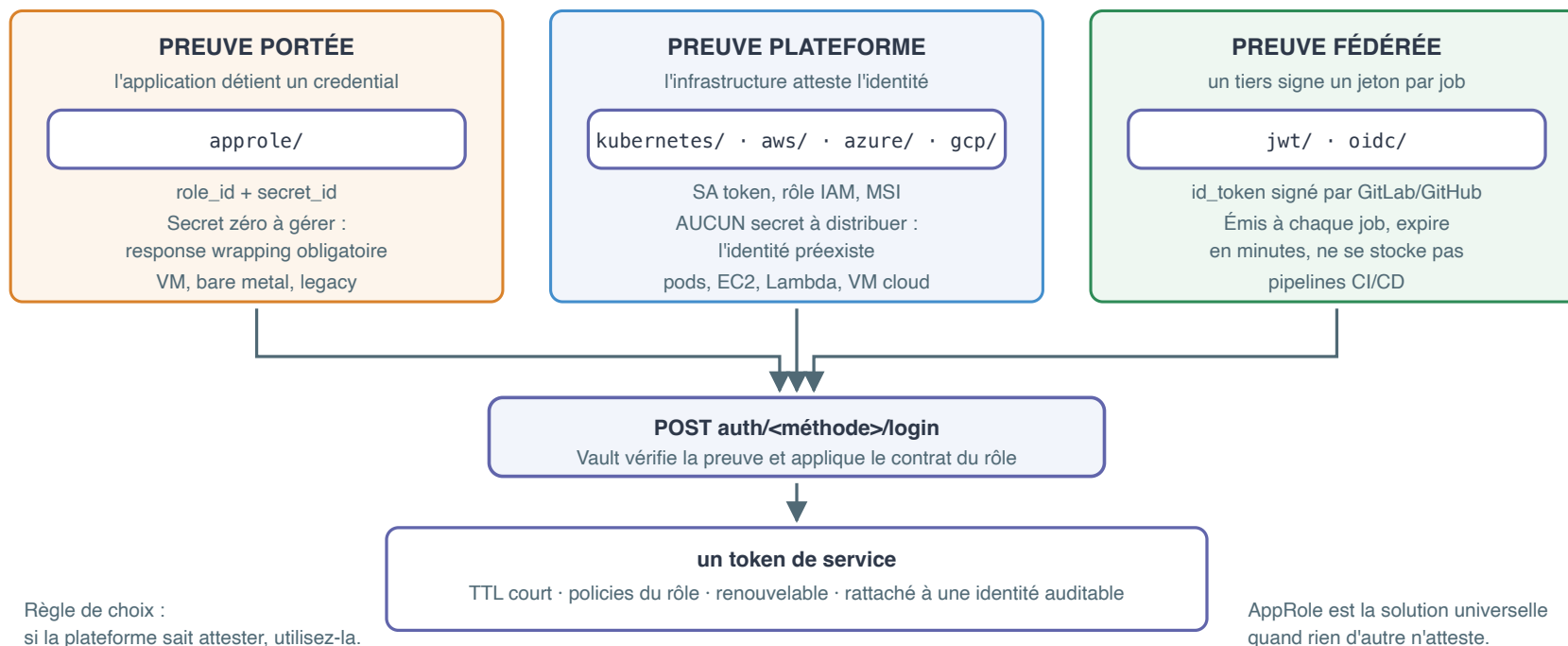
✓ Le token obtenu par login

l'application prouve son identité, Vault délivre un bail court



Panorama des auth methods côté application

« Qui es-tu ? Prouve-le. » — trois familles de preuves, un seul résultat



approle/ en détail ici — **kubernetes/** au module 5, JWT/CI au module 6.

2. AppRole en profondeur

AppRole : un compte de service, version Vault

Un **rôle** = l'identité déclarée d'une application, avec sa politique de délivrance de tokens :

```
auth/approle/role/paylink-api
|-- token_policies      : ce que ses tokens pourront faire
|-- token_ttl/max_ttl  : combien de temps
|-- secret_id_ttl       : durée de vie du credential
+-- secret_id_num_uses  : nombre de logins par credential
```

role_id et secret_id : deux natures

Le login exige un couple — pensez login / mot de passe pour machine :

	role_id	secret_id
Nature	Identifiant du rôle	Credential
Confidentialité	Faible (config, image, dépôt)	Haute — ne doit jamais traîner
Stabilité	Stable, généré une fois	Jetable : expire et s'use
Obtention	<code>read ../role-id</code>	<code>write -f ../secret-id</code>

Deux éléments, **deux canaux de distribution** : en voler un seul ne suffit pas.

Les quatre curseurs du rôle

Paramètre	Porte sur	Effet	PayLink
<code>secret_id_ttl</code>	secret_id	Péréemption du credential	60m
<code>secret_id_num_uses</code>	secret_id	Nb de logins puis destruction	5
<code>token_ttl</code>	token	Durée avant renew obligatoire	15m
<code>token_max_ttl</code>	token	Plafond absolu, renew refusé au-delà	1h

Lecture sécurité :

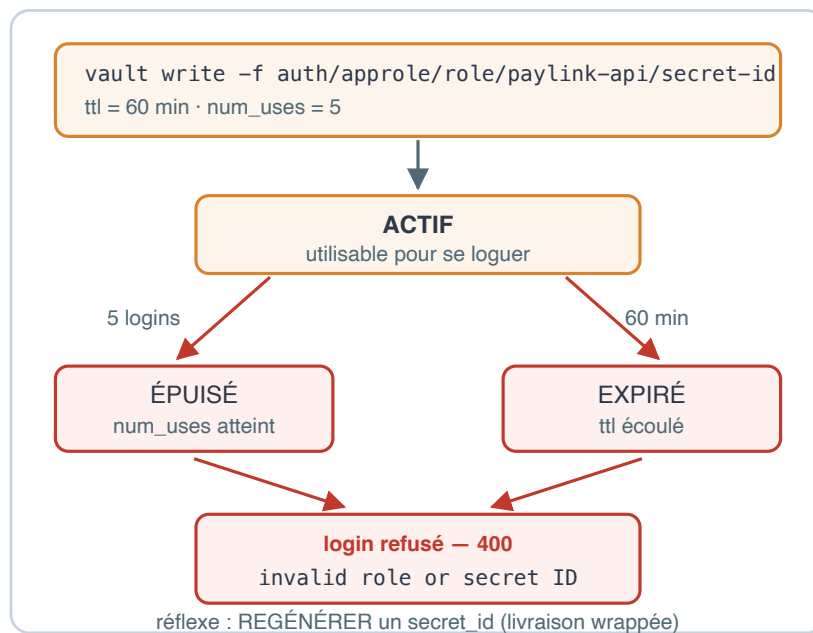
- Un secret_id volé n'est exploitable que 60 min **et** 5 logins max
- Un token volé meurt seul en 15 min sans renew, 1 h quoi qu'il arrive
- Les polices sont fixées par le rôle : l'app ne choisit pas ses droits

Ces valeurs se calibrent par service : MediTrack (santé) sera plus strict que PayLink — c'est l'exercice 2.1.

Cycle de vie : secret_id et token

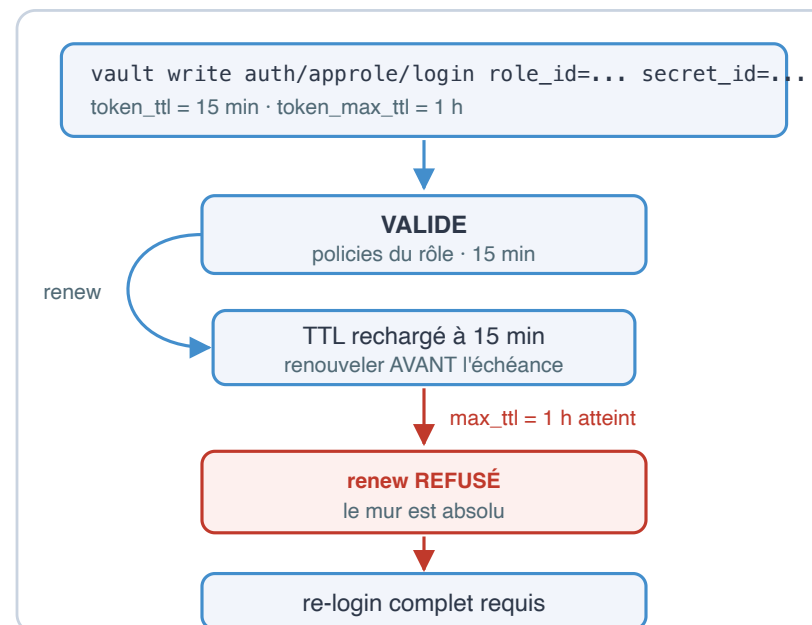
secret_id — le credential jetable

deux compteurs indépendants : une durée ET un nombre d'usages



token — le résultat du login

un TTL qu'on recharge, un plafond qu'on ne franchit pas



Deux compteurs indépendants, deux réflexes distincts.

Créer le rôle de l'API PayLink

```
v auth enable approle  
v write auth/approle/role/paylink-api \  
  token_policies=paylink-read \  
  token_ttl=15m token_max_ttl=1h \  
  secret_id_ttl=60m secret_id_num_uses=5
```

Un rôle = l'identité déclarée d'une application, avec sa politique de délivrance de tokens.

Relire ce que Vault a compris

```
v read auth/approle/role/paylink-api
```

```
bind_secret_id      true
secret_id_num_uses  5
secret_id_ttl       1h
token_max_ttl       1h
token_policies      [paylink-read]
token_ttl           15m
token_type          default
```

`bind_secret_id=true` (défaut) : le secret_id est exigé au login. Ne le désactivez qu'avec des contraintes CIDR en compensation.

Le role_id : l'identifiant du rôle

```
v read auth/approle/role/paylink-api/role-id
```

```
role_id      f33ea47d-7175-d0fb-5089-7ae531f967c2
```

Confidentialité **faible** : il peut vivre dans la config, l'image, le dépôt.
Stable, généré une fois pour le rôle.

Le secret_id : le credential

```
v write -f auth/approle/role/paylink-api/secret-id
```

```
secret_id          30c91e88-abcf-344b-e2fb-f7590aa08ac6
secret_id_accessor  a1370b04-446a-f084-8ba6-4e616e05973d
secret_id_num_uses  5
secret_id_ttl       1h
```

- Le `secret_id_accessor` : inventaire et révocation **sans connaître la valeur** (`.../secret-id-accessor/destroy`)
- Le `secret_id` vient de s'afficher dans un terminal... Gardez cette gêne en tête : c'est le sujet de la section 3

Login CLI : le token applicatif

```
v write auth/approle/login \  
  role_id=f33ea47d-7175-d0fb-5089-7ae531f967c2 \  
  secret_id=30c91e88-abcfc344b-e2fb-f7590aa08ac6
```

```
token                hvs.CAESILKWZtih3xx8pR0gDH4fvi7VM-PBxNMswkWPzz...  
token_accessor       UG1G00wwzvNptgveEfbJVAx3  
token_duration        15m  
token_renewable       true  
token_policies        ["default" "paylink-read"]  
token_meta_role_name  paylink-api
```

- 15 minutes, renouvelable, politiques du rôle + `default` (ajoutée à tout token : lookup-self, renew-self, cubbyhole)
- `token_meta_role_name` : l'audit log saura que c'est PayLink
- Ce login a consommé 1 des 5 usages du secret_id

Le même login en API REST – ce que fera le code

```
curl -s --request POST \  
  --data '{"role_id":"f33ea47d-...", "secret_id":"30c91e88-..."}' \  
  http://127.0.0.1:8200/v1/auth/approle/login | jq .auth
```

```
{  
  "client_token": "hvs.CAESIJVIPPUYHi-jv24wJA8E7fvsdZG-vv4Kc4eDcH8W...",  
  "token_policies": ["default", "paylink-read"],  
  "lease_duration": 900,  
  "renewable": true  
}
```

`lease_duration: 900` = `token_ttl` exprimé en secondes.

Puis lire, avec le token obtenu

Chaque lecture porte l'en-tête `X-Vault-Token` (module 1) :

```
curl -s --header "X-Vault-Token: $APP_TOKEN" \  
http://127.0.0.1:8200/v1/secret/data/paylink/api | jq .data.data
```

Deux requêtes HTTP — un login, puis des lectures.

C'est exactement ce que font tous les SDK Vault : ils n'ajoutent que la gestion du renouvellement et des erreurs réseau.

`lease_duration: 900` = `token_ttl` en secondes.



Démo 2.1

AppRole de bout en bout : du rôle au secret lu

`demos/module-02/demo-2-1-approle.md` — 10 min

Activer approle, créer `paylink-api`, se loguer en CLI puis en API REST, lire un secret avec le token applicatif... et se prendre un 403 en écriture.



Exercice 2.1

Un AppRole pour MediTrack

`exercices/module-02/exercice-2-1-approle-cli.md` — 25 min

Créer le rôle `meditrack-api` (policy fournie), se loguer, comparer les `token_policies`, constater l'isolation... et épuiser un `secret_id`.

3. Le secret zéro et le response wrapping

Le problème du secret zéro

Pour se loguer, PayLink a besoin du secret_id.
Pour livrer le secret_id, il faut un canal sûr...
donc un secret pour protéger le canal. **Récuratif.**

```

Opérateur / CI                                     API PayLink
-----
génère secret_id  ---- variable CI ? ----->  login
                  ---- fichier de config ? ---->
                  ---- e-mail / ticket ?! ----->
                  ^^^^^^^^^^^^^^^^^^^^^
                  chaque canal peut être lu, journalisé,
                  rejoué... SANS QUE PERSONNE NE LE SACHE

```

Le vrai danger n'est pas la copie — c'est la copie **indétectable** d'un credential **durable** (60 min, 5 logins).

On ne peut pas supprimer le secret zéro. On peut le rendre :
éphémère, à usage unique, observable.

Response wrapping : le principe

Demander à Vault de ne **pas répondre** — mais de ranger la réponse dans un cubbyhole et de rendre un jeton d'accès à cette réponse :

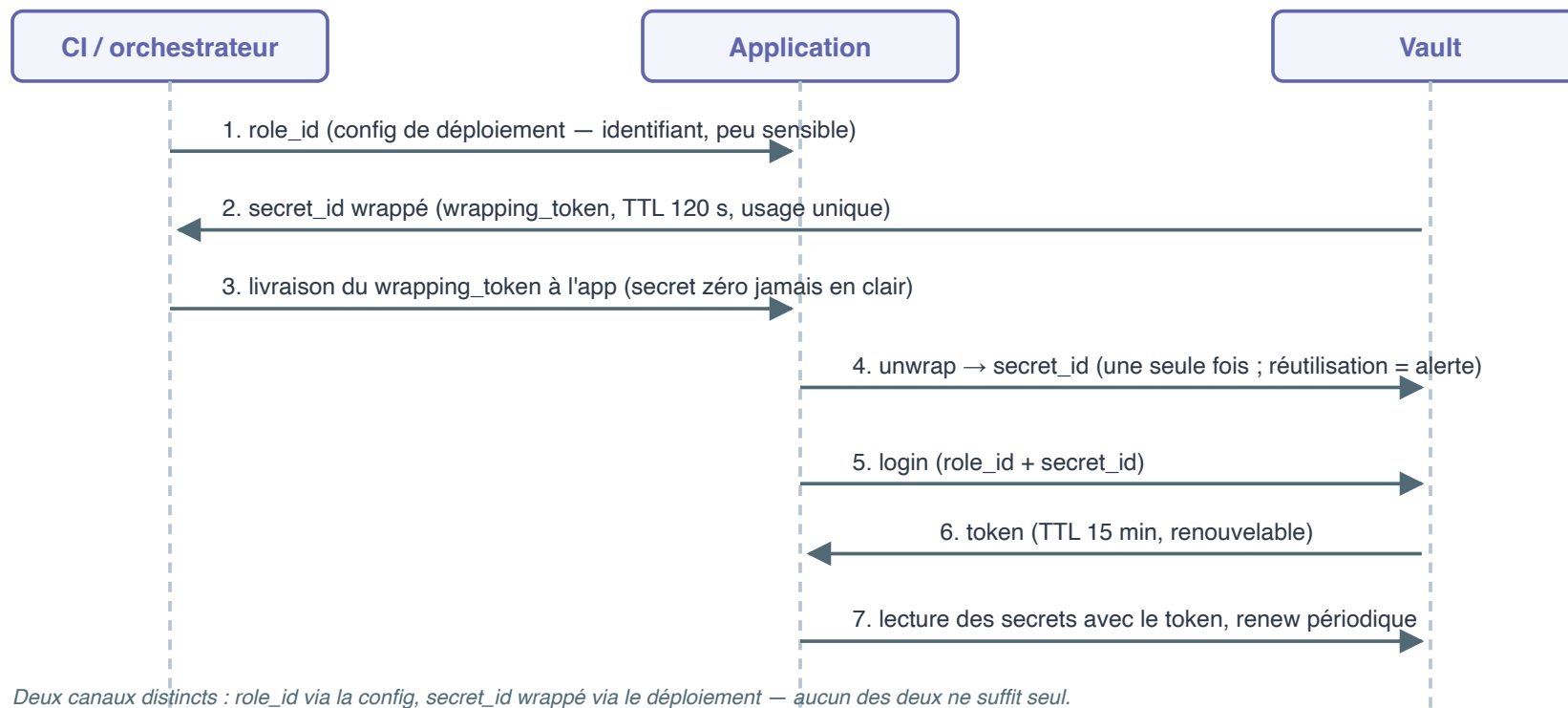
```
v write -wrap-ttl=120s -f auth/approle/role/paylink-api/secret-id
```

Propriétés du `wrapping_token` :

Propriété	Effet sécurité
TTL court (<code>wrap-ttl</code>)	Fenêtre d'attaque bornée : 120 s puis destruction
Usage unique	Le premier unwrap détruit le jeton et son contenu
<code>sys/wrapping/lookup</code>	Vérifiable sans consommer : chemin d'origine, date
Unwrap échoué = alerte	Si l'app reçoit 400, quelqu'un a ouvert avant elle

Celui qui génère ne voit jamais le `secret_id`. Celui qui transporte ne porte qu'un jeton jetable. Seul le destinataire ouvre — une fois.

La séquence complète



Le wrapping – côté opérateur

```
v write -wrap-ttl=120s -f auth/approle/role/paylink-api/secret-id
```

```
wrapping_token:          hvs.CAESIEN7La2PGV53Jk10brHheb3GZVLV_c...
wrapping_token_ttl:      2m
wrapping_token_creation_path: auth/approle/role/paylink-api/secret-id
wrapped_accessor:        2c306e7f-7e37-9352-5835-6e6d503d9dd7
```

`wrapped_accessor` : notez-le — c'est lui qui permettra de détruire le `secret_id` livré en cas d'incident. L'opérateur **ne voit jamais** le `secret_id`.

Le wrapping – côté application

Vérifier (sans consommer), puis ouvrir – une seule fois :

```
v write sys/wrapping/lookup token=hvs.CAESIEN7... # sans consommer  
v unwrap hvs.CAESIEN7...
```

```
secret_id          67f96c26-6a5d-ed8a-d3a3-89f202d8026b  
secret_id_accessor 2076972e-82da-4ecf-d6c3-ef9492c518d5  
secret_id_num_uses 5
```

Le `lookup` renseigne sur l'origine et la date **sans** détruire le jeton ;
seul l'`unwrap` le consomme.

Détection de compromission

Second unwrap du même jeton — sortie réelle :

```
Error unwrapping: Error making API request.  
  
URL: PUT http://127.0.0.1:8200/v1/sys/wrapping/unwrap  
Code: 400. Errors:  
  
* wrapping token is not valid or does not exist
```

Renversez le scénario : c'est **l'application légitime** qui reçoit ce 400 à son premier essai. Le paquet a été ouvert en transit.

Réponse à incident

À scripter, pas à improviser :

1. **Ne pas démarrer** / bloquer le déploiement
2. **Détruire** le secret_id volé : `.../secret-id-accessor/destroy`
3. **Auditer** le canal de livraison, régénérer une livraison propre

Sans wrapping : ce vol serait resté invisible.

Avec : il est **impossible à masquer** — c'est tout l'intérêt du jeton à usage unique.



Démo 2.2

Response wrapping : livrer le secret zéro sans l'exposer

`demos/module-02/demo-2-2-response-wrapping.md` — 8 min

Wrapper un secret_id, l'inspecter sans le consommer, l'unwrapper une fois...
et constater l'erreur 400 du second unwrap.

4. Auth Kubernetes et auth cloud : principes

Auth Kubernetes : la plateforme atteste

Sous Kubernetes, plus besoin de livrer un credential : chaque pod possède déjà une identité — le token de son **ServiceAccount**, monté par kubelet.

Pod paylink-api -----	Vault -----	API Kubernetes -----
1. lit son SA token (/var/run/secrets/...)		
2. POST auth/kubernetes/login ---> { role, jwt=SA token }		
	3. TokenReview: "ce jwt est-il valide ? qui ?"	--->
	4. "oui: system:serviceaccount: paylink/paylink-api"	<---
<--- 5. token Vault (policies du rôle, bound au namespace + SA)		

Ce que l'auth Kubernetes change

- Le rôle Vault **borne** : `bound_service_account_names` + `namespaces`
- Le secret zéro disparaît : c'est kubelet qui a « livré » l'identité
- Rotation, expiration, révocation du SA token : gérées par Kubernetes

C'est le même modèle que l'auth AWS/Azure : *la plateforme atteste, Vault vérifie.*

Mise en pratique complète (agent, injector, VSO) : **module 5.**

Auth cloud : AWS IAM, Azure MSI

Même idée : la preuve d'identité est **fournie par la plateforme**, jamais livrée par vous.

	AWS (<code>aws</code> , type iam)	Azure (<code>azure</code>)
Identité de la charge	Rôle IAM (instance profile, Lambda, ECS task)	Managed Identity (MSI) de la VM / app
Preuve présentée	Requête STS <code>GetCallerIdentity</code> signée SigV4	JWT MSI obtenu sur l'IMDS local
Vérification Vault	Rejoue la requête vers STS et lit l'ARN	Valide le JWT + interroge Azure
Rôle Vault borne	ARN de rôle/compte, région	subscription, resource group, VM

Auth cloud : ce qu'il faut retenir

Points communs — et c'est le message :

- **Credential** : aucun à distribuer ; il **préexiste** dans la plateforme
- **Rotation** : c'est le problème du fournisseur, pas le vôtre
- **Rôle Vault** : il traduit une identité plateforme en policies Vault

GCP, Alibaba, OCI : mêmes patterns.

CI/CD (JWT/OIDC) : c'est le **module 6**.

Choisir sa méthode : tableau décisionnel

Critère	AppRole	Kubernetes	Cloud (IAM / MSI)
Où tourne l'app ?	VM, bare metal, legacy, hors plateforme	Pods K8s / OpenShift	EC2, Lambda, AKS, VM cloud
Secret zéro	À gérer (wrapping obligatoire)	Aucun (SA token monté)	Aucun (identité plateforme)
Prérequis	Aucun – universel	Accès TokenReview	Vault joignable + API cloud
Rotation credential	À orchestrer (secret_id)	Automatique (K8s)	Automatique (cloud)
Granularité identité	1 rôle / service	SA + namespace	Rôle IAM / MSI
Effort d'exploitation	Le plus élevé	Faible	Faible

L'arbre de décision

```
L'app tourne-t-elle sur une plateforme qui sait attester son identité ?  
|  
OUI                                     NON  
|                                     |  
K8s ? -> auth kubernetes             approle + response wrapping  
AWS ? -> auth aws (iam)               (+ CIDR bounds si réseau maîtrisé)  
Azure ? -> auth azure  
Pipeline CI ? -> jwt/oidc (module 6)
```

Règle : AppRole est le choix **par défaut quand rien d'autre n'atteste** — pas le choix par habitude.

5. Vivre avec son token

Renouveler... jusqu'au plafond

```

login                renew                renew                max_ttl = 1h
|                    |                    |                    |
v                    v                    v                    v
|===== 15m =====>|== recharge ==>|===== ... =====X
t0                  t0+13m                t0+27m                t0+1h
                    TTL 15m                TTL 15m                renew REFUSÉ
                                                => re-login

```

```

vault token renew      # avec VAULT_TOKEN = token applicatif

```

```

token_duration        15m          <- rechargé (900s via l'API renew-self)

```

Les trois réflexes du renouvellement

- Renouvelez **avant** l'échéance — marge conseillée : 1/3 du TTL restant
- `token_max_ttl` est un **mur** : prévoyez le **re-login** dans le code, donc un `secret_id` encore valide ou une nouvelle livraison wrappée
- Un renew qui échoue n'est pas une erreur à retenter en boucle : c'est le **signal de re-login**

C'est précisément ce que Vault Agent (module 5) automatise pour vous.

Le pattern applicatif complet

OPÉRATEUR / CI

1. write -wrap-ttl=120s
.../secret-id
2. livre wrapping_token + role_id
(fichier, env, annotation...)

APPLICATION (démarrage)

3. lookup du wrapping token (contrôle)
4. unwrap -> secret_id [1 fois]
5. login(role_id, secret_id) -> token 15m
6. kv get secret/paylink/api
7. boucle : sleep ; token renew
|
+-- renew refusé (max_ttl / expiré)
-> re-login (5)

C'est exactement le contenu de `code/module-02/` :
`deploy-wrap.sh` (étapes 1-2) et `app-unwrap-login.sh` (étapes 3-7).

À retenir : ce pattern se **scripte une fois** puis s'industrialise —
Vault Agent (module 5) le fait pour vous, déclarativement.



TP 2

Livrer le secret_id à l'API PayLink sans jamais l'exposer

`tp/module-02/tp-2-approle-wrapping.md` — 45 min

Un script opérateur qui wrappe, une application qui unwrappe, se logue, lit ses secrets et entretient son token. Puis vous jouez l'attaquant : interception, alerte, destruction du secret_id volé, relivraison.

Récap du module

- **Jamais de token statique** : il fuit, ne tourne pas, n'a pas d'identité. Le token s'obtient par un login, il ne se distribue pas
- **AppRole** : role_id (identifiant) + secret_id (credential jetable : TTL + usages) → token court, scopé par les politiques du rôle
- **Secret zéro** : insoluble, mais transformable — le response wrapping livre un jeton à usage unique, TTL court, dont le vol se **détecte** (unwrap en 400 = alerte, destroy par accessor, relivraison)
- **Choix de méthode** : si la plateforme atteste (K8s, AWS, Azure), utilisez-la ; AppRole est la solution universelle quand rien n'atteste
- **Cycle de vie** : renew avant l'échéance, re-login à `token_max_ttl`

Cheatsheet : `cheatsheet/module-02.md`

Module suivant : concevoir les politiques que portent ces tokens.

Quiz – questions 1 et 2

Question 1. Citez les trois défauts rédhibitoires d'un token statique placé dans la configuration d'une application.

Question 2. Dans AppRole, quelle est la différence de nature entre le `role_id` et le `secret_id`, et quelle conséquence sur la façon de les distribuer ?

Quiz — questions 3 et 4

Question 3. Un secret_id créé avec `secret_id_num_uses=5` vient de servir à un 5e login. Que devient-il, et que renvoie un 6e login ?

Question 4. Un rôle est configuré avec `token_ttl=15m` et `token_max_ttl=1h`. Que permet le renew entre ces deux bornes, et que se passe-t-il une fois le plafond atteint ?

Quiz – questions 5 et 6

Question 5. Qu'appelle-t-on le problème du « secret zéro », et pourquoi ne peut-on pas le « résoudre » au sens strict ?

Question 6. Quelles sont les deux propriétés du wrapping token qui en font une réponse acceptable au secret zéro ?

Quiz — questions 7 et 8

Question 7. Au démarrage, votre application reçoit un code 400 (`wrapping token is not valid or does not exist`) lors de l'unwrap de sa livraison. Que devez-vous en conclure et quelles actions déclencher ?

Question 8. Sur quel mécanisme repose la méthode d'authentification `kubernetes` de Vault pour vérifier l'identité d'un pod ?

Quiz — questions 9 et 10

Question 9. Votre équipe déploie un service sur des VM AWS EC2 et un batch legacy sur un serveur physique. Quelle méthode d'authentification recommandez-vous pour chacun, et pourquoi ?

Question 10. Le token de votre application expire dans 2 minutes. Quelles sont vos deux options, et qu'est-ce qui détermine laquelle s'applique ?

Questions ?

Module suivant : **Policies et moindre privilège**
Concevoir, templater et revoir les policies que portent vos tokens.

Module 3

Policies et moindre privilège

HashiCorp Vault — Utilisation & Intégration

Objectifs du module

À la fin de ce module, vous serez capable de :

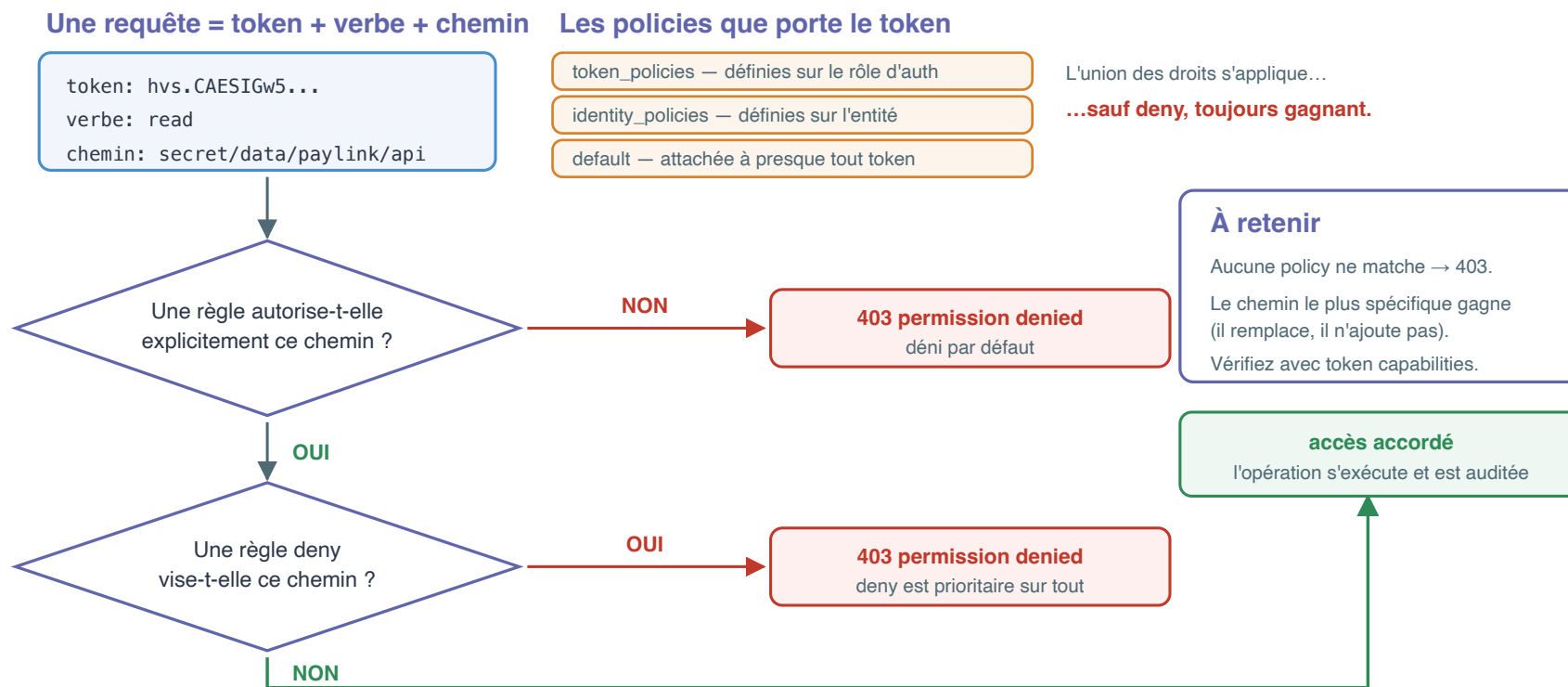
- Expliquer le modèle d'autorisation de Vault : déni par défaut, union des politiques portées par le token
- Écrire une policy HCL correcte : paths, capabilities, globs, cas KV v2
- Vérifier une policy avec `vault token capabilities` avant toute mise en prod
- Relier un token à une identité : entities, aliases, `identity_policies`
- Écrire une policy templatée (`{{identity.entity.name}}`) qui cloisonne N services avec un seul gabarit
- Concevoir et relire des politiques selon le moindre privilège par service

Plan

1. Le modèle d'autorisation
2. Syntaxe HCL : paths et capabilities
3. Le cas particulier KV v2
4. Vérifier : `token capabilities` et le 403
5. Identité : entities, aliases
6. Politiques templatées – et leurs pièges réels
7. Concevoir des politiques par service
8. Récap et quiz

1. Le modèle d'autorisation

Déni par défaut



Rien qui matche : **403**. Un **deny** explicite gagne sur tout.

Le token porte des policies

```
$ vault token create -policy=paylink-read -ttl=1h
```

```
policies [default paylink-read]
```

- Un token porte **une liste de policies**, figée à sa création
- Plusieurs policies : c'est **l'union** des droits qui s'applique — sauf **deny**, toujours gagnant

Deux politiques particulières

- **default** — attachée à presque tous les tokens.
Elle n'accorde presque rien : **lookup-self**, **renew-self**, l'accès au cubbyhole du token. C'est le socle minimal pour qu'un token puisse s'auto-gérer.
- **root** — tout est permis, sur tous les chemins.
Réservée à l'administration d'urgence, **jamais à une application**.
Un token root en production est un incident de sécurité en attente.

D'où viennent les policies d'un token ?

```
login approle --> +-----+
                   | token_policies | définies sur le rôle
                   | (paylink-read) | d'authentification
                   +-----+
                   | identity_policies | définies sur l'ENTITÉ
                   | (service-scoped) | (identité transverse)
                   +-----+
                   | default           |
                   +-----+
```

Le même login peut donc recevoir des droits de **deux sources** :
le rôle d'auth (`token_policies`) et l'identité (`identity_policies`).

Nous verrons l'identité en détail dans la section 5.

2. Syntaxe HCL

Anatomie d'une policy

```
# Lecture seule des secrets applicatifs PayLink
path "secret/data/paylink/*" {
  capabilities = ["read"]
}

path "secret/metadata/paylink/*" {
  capabilities = ["read", "list"]
}
```

- Un fichier `.hcl` = une suite de blocs `path`
- Chaque bloc : un **chemin** (avec globs) + des **capabilities**
- Le chemin est celui de **l'API REST** (pas celui de la CLI `kv`)

Les capabilities

Capability	Verbe HTTP	Usage
<code>create</code>	POST/PUT (chemin inexistant)	créer un secret
<code>read</code>	GET	lire
<code>update</code>	POST/PUT (chemin existant)	modifier
<code>patch</code>	PATCH	modification partielle (KV v2)
<code>delete</code>	DELETE	supprimer
<code>list</code>	LIST	énumérer les clés
<code>deny</code>	—	tout refuser, prioritaire
<code>sudo</code>	—	chemins protégés (<code>sys/...</code>)

En pratique, écriture applicative = `["create", "update"]` (le chemin existe ou non).

Globs : * et +

```
path "secret/data/paylink/*"      # tout le sous-arbre paylink/  
path "secret/data/+/config"      # config de N'IMPORTE QUEL service  
path "secret/data/paylink/api"    # ce chemin exactement
```

- * : uniquement **en fin de chemin**, matche tout le sous-arbre
- + : exactement **un segment**, n'importe où dans le chemin

Priorité : le chemin le plus spécifique gagne

```
path "secret/data/paylink/*"      { capabilities = ["read"] }  
path "secret/data/paylink/webhooks" { capabilities = ["create", "read", "update"] }
```

Sur `paylink/webhooks`, c'est la **seconde** règle qui s'applique.

Elle est plus spécifique : elle **remplace** la première, elle ne s'y ajoute pas. Le token n'aura donc pas les droits cumulés des deux règles.

3. Le cas particulier KV v2

La policy vise le chemin API, pas le chemin CLI

Piège n° 1, responsable de la moitié des 403 « incompréhensibles » :

CLI		API réelle (= policy)
-----		-----
<code>vault kv get secret/x</code>	->	<code>GET secret/data/x</code>
<code>vault kv put secret/x</code>	->	<code>POST secret/data/x</code>
<code>vault kv list secret/</code>	->	<code>LIST secret/metadata/</code>
<code>vault kv delete secret/x</code>	->	<code>DELETE secret/data/x (soft)</code>
<code>vault kv undelete ...</code>	->	<code>POST secret/undelete/x</code>
<code>vault kv destroy ...</code>	->	<code>POST secret/destroy/x</code>
<code>vault kv metadata get</code>	->	<code>GET secret/metadata/x</code>

Une policy sur `secret/paylink/*` **ne matche rien** en KV v2 :
il faut `secret/data/paylink/*`, `secret/metadata/paylink/*`, etc.

L'arbre complet d'un secret KV v2

Ce que vous tapez (CLI)

`vault kv get secret/paylink/api`
chemin logique — la CLI complète
le chemin pour vous

Ce que l'API reçoit (KV v2)

`GET /v1/secret/data/paylink/api`
chemin réel — c'est LUI que la policy doit matcher
l'oublier dans une policy KV v2 = 403 garanti

insère
data/

L'arbre complet d'un secret KV v2 — une famille d'opérations par préfixe

secret/
mount KV v2

data/paylink/api
lire et écrire les données · read/create/update

metadata/paylink/api
versions, timestamps, list · read/list

delete/ · undelete/
suppression réversible de versions précises

destroy/
effacement DÉFINITIF — jamais accordé à une app

Conséquence : le moindre privilège

On accorde la lecture des données
sans le droit de lister.

On accorde l'écriture
sans la suppression.

Chaque famille d'opérations ayant
son préfixe, la policy découpe au verbe près.

Un préfixe par famille : la policy découpe au verbe près.

Gérer les policies : écrire et relire

```
$ vault policy write paylink-read /tmp/paylink-read.hcl  
Success! Uploaded policy: paylink-read
```

```
$ vault policy read paylink-read  
# Lecture seule des secrets applicatifs PayLink  
path "secret/data/paylink/*" {  
  capabilities = ["read"]  
}
```

Gérer les policies : lister

```
$ vault policy list  
default  
paylink-read  
root
```

Une policy est un **objet nommé**, versionné par vos soins dans git — Vault, lui, ne garde que la dernière version.

D'où la règle : le fichier `.hcl` fait foi, il est revu comme du code.

4. Vérifier avant de croire

token capabilities : l'outil n° 1

Le réflexe à acquérir : **demander à Vault ce qu'un token peut faire** sur un chemin, au lieu de le deviner en relisant le HCL.

```
$ vault token capabilities secret/data/paylink/api  
read  
  
$ vault token capabilities secret/data/rh/paie  
deny
```

Sorties réelles du lab : le token porte `paylink-read`, il lit PayLink, tout le reste est `deny`.

Tester le token d'un autre

Variante administrateur :

```
$ vault token capabilities <token> secret/data/paylink/api
```

Trois usages quotidiens :

- en **revue de policy**, avant la mise en production
- en **débogage de 403**, pour trancher entre policy et token
- dans les **TP** de cette formation, pour prouver le cloisonnement

Lire un 403

```
$ vault kv put secret/paylink/api hack=1
Error writing data to secret/data/paylink/api: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/secret/data/paylink/api
Code: 403. Errors:

* 1 error occurred:
    * permission denied
```

Ce que dit ce message — et ce qu'il ne dit pas :

- l'**URL réelle** appelée : c'est elle que la policy doit matcher
- `permission denied` : token invalide **ou** droit manquant — Vault ne précise volontairement pas (ne rien apprendre à un attaquant)

Démo 3.1

Écrire une policy HCL et la prouver

- Écrire et charger `paylink-read`
- Créer un token restreint
- `token capabilities` : `read` vs `deny`
- Lecture qui passe, écriture en 403

`demos/module-03/demo-3-1-policies-hcl.md`

5. Identité : entities et aliases

Pourquoi une couche d'identité ?

Un même service peut se présenter par plusieurs portes : AppRole aujourd'hui, Kubernetes demain, JWT dans le pipeline.

```

+-----+
auth approle ---+ | ENTITY paylink-api |
(alias:      +-->| policies:      |
  role_id)   | |   service-scoped |
              | | (identity_policies) |
auth kubernetes -+ +-----+
(alias: SA)

```

- **Entity** : l'identité logique du service, unique et nommée
- **Alias** : le lien entre une méthode d'auth et cette entité
- Les politiques posées sur l'entité suivent le service **quelle que soit la porte d'entrée** → `identity_policies` sur le token

Piège réel n° 1 : l'entité auto-créée

Au **premier login** d'une méthode d'auth, si aucun alias ne correspond, Vault **crée une entité automatiquement** (nom `entity_xxxx`).

Session réelle du lab — deux entités au lieu d'une :

```
$ vault list identity/entity/id      (détail par id)
1b8d2abb  name=entity_cde8b386  policies=[]                  aliases=[ 'f33ea47d-7175-d0fb-5' ]
c118d079  name=paylink-api      policies=['service-scoped']  aliases=['paylink-api']
```

Pourquoi ça produit un 403

- `entity_cde8b386` : créée par Vault **au premier login** (alias = role_id). C'est elle que les logins utilisent — et elle n'a **aucune policy**.
- `paylink-api` : créée à la main **après coup**. Jamais utilisée par les logins, d'où un 403 malgré une config « qui a l'air bonne ».

Et en voulant corriger naïvement, l'erreur suivante :

```
* alias with combination of mount accessor and name already exists
```


Piège réel n° 2 : l'alias AppRole porte le role_id

Pour la méthode `approve`, le **nom d'alias** attendu est le `role_id` (un UUID) — pas le nom du rôle.

C'est contre-intuitif, et c'est la cause n° 1 des politiques templatées qui « ne s'appliquent pas ».

Les deux stratégies qui fonctionnent

A. Le service ne s'est jamais connecté — créer l'identité AVANT :

```
vault write identity/entity name=meditrack-api policies=service-scoped
vault write identity/entity-alias name=<ROLE_ID> \
    canonical_id=<ENTITY_ID> mount_accessor=<ACCESSOR_APPROLE>
```

B. Le service s'est déjà connecté — adopter l'entité auto-crée :

```
vault write identity/entity/id/<ID_AUTO> name=paylink-api policies=service-scoped
```

Vérification après login : `identity_policies ["service-scoped"]` sur le token.

6. Politiques templatées

Un gabarit pour N services

```
# Chaque service ne voit que son propre espace
path "secret/data/services/{{identity.entity.name}}/*" {
  capabilities = ["read", "list"]
}
```

- Évalué à **chaque requête**, avec l'identité du token appelant
- `paylink-api` lit `services/paylink-api/*` ;
`meditrack-api` lit `services/meditrack-api/*` — même policy
- Dix services = **un seul fichier** à maintenir, zéro copier-coller

Condition de fonctionnement : le token doit être **rattaché à une entité nommée** (section 5) — sinon le gabarit ne résout rien et c'est 403.

Paramètres de templating utiles

Paramètre	Contenu
<code>identity.entity.name</code>	nom de l'entité
<code>identity.entity.id</code>	id de l'entité
<code>identity.entity.metadata.<clé></code>	métadonnée posée sur l'entité
<code>identity.entity.aliases.<accessor>.name</code>	nom d'alias pour une méthode

Exemple avec métadonnée (`env=prod` posée sur l'entité) :

```
path "secret/data/{{identity.entity.metadata.env}}/{{identity.entity.name}}/*" {
  capabilities = ["read"]
}
```

La preuve par le cloisonnement

Sorties réelles du lab, après correction des deux pièges :

```
login approle (paylink-api)
  token_policies      ["default" "paylink-read"]
  identity_policies    ["service-scoped"]

--- lecture de SON espace
$ vault kv get -field=db_host secret/services/paylink-api/config
pg.internal
```

Le token porte bien la policy templâtée, via son **entité**.

Et chez le voisin : 403

```
--- lecture de l'espace de l'AUTRE service  
$ vault kv get secret/services/meditrack-api/config  
Code: 403. Errors:  
    * permission denied
```

Une **seule** policy templatée, deux services cloisonnés : le gabarit `{{identity.entity.name}}` résout vers `paylink-api`, donc `meditrack-api` ne matche aucune règle → déni par défaut.

C'est exactement le scénario que vous rejouerez au **TP 3**.

Démo 3.2

Policies templatées : pourquoi ça fait 403 ?

Débogage scénarisé, erreurs réelles comprises :

- La policy templatée « intuitive »... et son 403
- Diagnostic : lister les entités, découvrir l'auto-crée
- Les deux corrections possibles
- Cloisonnement prouvé

```
demos/module-03/demo-3-2-policies-templatees.md
```




Exercice 3.1

Écrire une policy de moindre privilège

Traduire un besoin métier en HCL :

- lecture de tout `paylink/`
- écriture **uniquement** sur `paylink/webhooks`
- listing autorisé, suppression **interdite**
- preuve par `token capabilities` sur 6 chemins

~30 min — `exercices/module-03/exercice-3-1-ecrire-une-policy.md`

7. Concevoir des politiques par service

Une policy par service

Règles de conception éprouvées :

- **Une policy = un service** (ou un gabarit templaté = une famille de services) ; jamais de policy « fourre-tout » partagée
- **Nommage prévisible** : `<service>-<usage>` (`paylink-read`, `paylink-cicd`) — le nom dit ce que la policy accorde
- **Partir de zéro droit** et ajouter, jamais partir de `*` et retrancher
- Le fichier HCL vit **dans git**, revu comme du code
- Commentaire en tête : à quoi sert la policy, pour qui, depuis quand

```
services/      policies/
paylink-api    -> paylink-read.hcl  (+ service-scoped.hcl templatée)
meditrack-api  -> meditrack-read.hcl (ou le même gabarit)
pipeline CI    -> paylink-cicd.hcl
```

Revue de policy : les signaux qui doivent alerter

Signal	Pourquoi c'est suspect
path "*" ou secret/*	accès à tout, moindre privilège violé
capabilities = [...tout...]	personne n'a besoin des 7
sudo	réservé à l'admin, jamais applicatif
`secret/delete	destroy/...` accordé
policy partagée entre services	fuite latérale en cas de compromission
pas de deny sur l'ultra-sensible	la défense en profondeur manque

Le **deny** explicite est votre garde-fou : il gagne sur **toutes** les autres policies du token, y compris futures.

 TP 3

Cloisonner PayLink et MediTrack

- **Une** policy templatée pour les deux services
- PayLink : adopter l'entité auto-crée
- MediTrack : créer l'identité avant le premier login
- Tests croisés : chacun chez soi, 403 chez le voisin
- Grille d'évaluation et teardown inclus

~45 min — `tp/module-03/tp-3-cloisonner-les-services.md`

Récap du module

- **Déni par défaut** : tout ce qui n'est pas explicitement accordé est refusé ; `deny` explicite gagne sur tout
- Une policy = des blocs `path` + `capabilities` ; le chemin le plus spécifique s'applique
- **KV v2** : la policy vise `secret/data/...`, `secret/metadata/...` — jamais `secret/...` directement
- `vault token capabilities` : vérifier, ne pas deviner
- **Entity + alias** = identité transverse ; attention à l'entité auto-créeée au premier login et au nom d'alias AppRole = `role_id`
- Policy **templâtée** `{{identity.entity.name}}` : un gabarit, N services cloisonnés
- Conception : une policy par service, nommage clair, revue en git, zéro wildcard large, zéro `sudo` applicatif

Quiz – questions 1 et 2

Question 1 – Un token porte deux policies : l'une accorde `read` sur `secret/data/paylink/api`, l'autre contient `deny` sur ce même chemin. Que se passe-t-il à la lecture, et pourquoi ?

Question 2 – Votre policy contient `path "secret/paylink/*"` et pourtant `vault kv get secret/paylink/api` renvoie 403. Qu'avez-vous oublié ?

Quiz — questions 3 et 4

Question 3 — Quelle est la différence entre les globs `*` et `+` dans un chemin de policy ?

Question 4 — Deux règles matchent `secret/data/paylink/webhooks` : `secret/data/paylink/*` (read) et `secret/data/paylink/webhooks` (create, read, update). Lesquelles de ces capabilities s'appliquent ?

Quiz — questions 5 et 6

Question 5 — Quelle commande permet de savoir ce qu'un token peut faire sur un chemin donné, sans tenter l'opération ?

Question 6 — `vault kv list secret/paylink` échoue en 403 alors que la policy accorde `read` sur `secret/data/paylink/*`. Quel chemin et quelle capability manquent ?

Quiz — questions 7 et 8

Question 7 — Qu'est-ce qu'une entity et un alias, et laquelle des deux porte les `identity_policies` ?

Question 8 — Au premier login AppRole d'un service, aucune entité n'a été préparée. Que fait Vault, et quelle conséquence pour une policy templatée sur `{{identity.entity.name}}` ?

Quiz — questions 9 et 10

Question 9 — Pour la méthode AppRole, quel doit être le **nom** d'un alias d'entité pour que les logins soient rattachés à la bonne entité ?

Question 10 — Citez trois signaux d'alerte lors d'une revue de policy applicative.

Fin du module 3

Fin du jour 1

Demain : secrets dynamiques, chiffrement as a service,
Kubernetes et CI/CD

Jour 2

Intégrer Vault

Module 4

Secrets dynamiques et chiffrement as a service

HashiCorp Vault – Utilisation & Intégration

Jour 2 – Durée : 2 h 30

Objectifs du module

À la fin de ce module, vous serez capable de :

- Expliquer les limites des secrets statiques partagés et le principe du secret dynamique
- Configurer le moteur `database` pour générer des comptes PostgreSQL éphémères
- Gérer le cycle de vie d'un lease : renouvellement, révocation, expiration
- Chiffrer et déchiffrer des données via le moteur `transit`, sans clé dans l'application
- Faire tourner une clé de chiffrement et re-chiffrer l'existant sans exposer le clair
- Choisir entre KV statique, database dynamique et transit selon le besoin

Plan

1. Des secrets statiques aux secrets dynamiques
2. Le moteur `database` avec PostgreSQL
3. Le cycle de vie des leases
4. Le moteur `transit` : chiffrement as a service
5. Choisir le bon moteur
6. TP 4 – PayLink passe au dynamique et chiffre les cartes

Fil rouge : l'API PayLink parle à PostgreSQL et doit chiffrer des numéros de cartes.

1. Des secrets statiques aux secrets dynamiques

Le problème du compte partagé

Aujourd'hui chez PayLink : un seul compte `paylink_app` / `S3cret!` dans PostgreSQL, stocké en KV, copié partout.

- **Partagé** : l'API, les jobs batch, les développeurs, le pipeline CI utilisent le même compte
- **Éternel** : le mot de passe n'a pas changé depuis 18 mois — personne n'ose le faire tourner
- **Anonyme** : dans les logs PostgreSQL, tout le monde est `paylink_app` — qui a fait ce `DELETE` ?
- **Irrévocable finement** : révoquer le compte = couper *tous* les consommateurs d'un coup
- Une fuite (dump de config, log, laptop volé) reste exploitable **indéfiniment**

Un secret statique en KV résout le stockage, pas la rotation ni l'imputabilité.

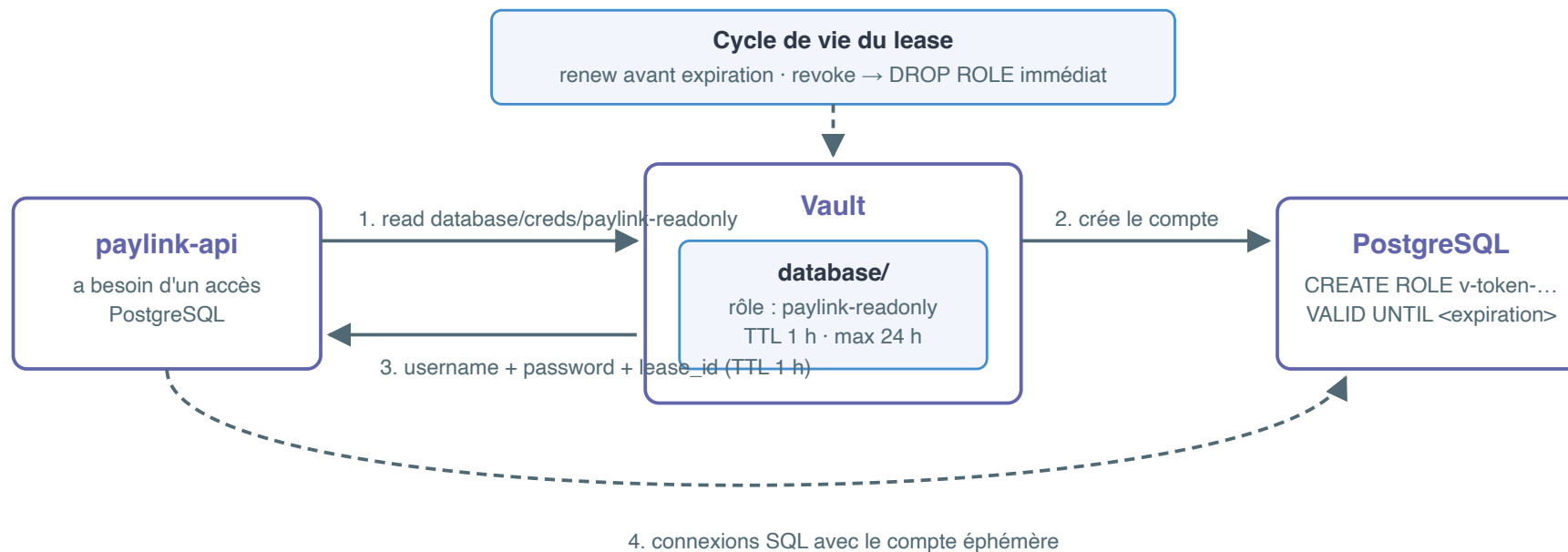
Le principe du secret dynamique

Vault ne stocke plus le secret : il le **fabrique à la demande**, pour chaque consommateur.

Propriété	Conséquence
Un compte par consommateur	Chaque instance de l'API a ses propres identifiants
TTL intégré	Le compte expire tout seul (lease), inutile de « penser à » le supprimer
Révocable unitairement	Compromission → on révoque ce lease, les autres continuent
Audité	Chaque génération est tracée : qui, quand, avec quel token

Le secret devient un **consommable jetable**, plus un patrimoine à protéger 18 mois.

Vue d'ensemble



2. Le moteur **database** avec PostgreSQL

Architecture du moteur database

Trois objets à configurer, une seule lecture côté application :

```
database/config/paylink-pg      <- COMMENT se connecter (compte de gestion, plugin)
database/roles/paylink-readonly <- QUOI créer (SQL de création, TTL)
database/creds/paylink-readonly <- lecture = Vault CRÉE un compte et vous le donne
```

- **config/** : la **connexion** — Vault détient un compte PostgreSQL *de gestion* capable de **CREATE ROLE**
- **roles/** : le **modèle** de compte à fabriquer — instructions SQL paramétrées
- **creds/** : le point de consommation — chaque **read** fabrique un compte neuf

L'application ne voit que **database/creds/<role>** : c'est tout ce que sa policy autorise.

Configurer la connexion

```
v secrets enable database

v write database/config/paylink-pg \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@postgres:5432/paylink?sslmode=disable" \
  allowed_roles="paylink-readonly,paylink-dml" \
  username="postgres" \
  password="rootpg"
```

- `{{username}}` / `{{password}}` : gabarits remplacés par le compte de gestion — le mot de passe n'apparaît **jamais** en clair dans l'URL stockée
- `username` / `password` : le compte de **gestion** (ici `postgres`) — c'est lui que Vault utilisera pour créer les comptes éphémères
- `allowed_roles` : liste blanche des rôles Vault autorisés à utiliser cette connexion

Relire `database/config/paylink-pg` ne renvoie pas le mot de passe de gestion.

Définir un rôle : les creation_statements

```
v write database/roles/paylink-readonly \  
  db_name=paylink-pg \  
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}'  
    VALID UNTIL '{{expiration}}';  
    GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"{{name}}\";" \  
  default_ttl=1h \  
  max_ttl=24h
```

- **creation_statements** : le SQL **que vous écrivez** — Vault injecte **{{name}}**, **{{password}}**, **{{expiration}}**
- **VALID UNTIL** : ceinture et bretelles — même si Vault ne révoque pas, PostgreSQL refusera le login après l'échéance
- **GRANT SELECT** : le compte fabriqué est **moins privilégié** que le compte de gestion — moindre privilège appliqué au niveau SQL
- **default_ttl=1h** : durée du lease ; **max_ttl=24h** : durée maximale, renouvellements compris

Consommer : read database/creds/<role>

```
v read database/creds/paylink-readonly
```

Key	Value
---	----
lease_id	database/creds/paylink-readonly/NSmKt6GBHjJiWg0sR6hE1Bm0
lease_duration	1h
lease_renewable	true
password	0e6fs1-oB2IZuErFrvb1
username	v-token-paylink--UyagA0CDoC56SJlJpWup-1783451428

- Chaque **read** = un **nouveau compte** PostgreSQL, un **nouveau lease**
- **v-token-paylink--...** : le nom encode la méthode d'auth, le rôle et un horodatage — traçable dans les logs PostgreSQL
- Deux instances de l'API → deux **read** → deux comptes distincts, révocables séparément

La preuve côté PostgreSQL

Ce ne sont pas des comptes « virtuels » : ils existent réellement dans `pg_user`.

```

      username                               |      valuntil
-----+-----
v-token-paylink--UyagAOCDoC56SJlJpWup-1783451428 | 2026-07-07 20:10:33+00
v-token-paylink--3oID1SPcRYEp3LHIYkhu-1783451429 | 2026-07-07 20:10:34+00
(2 rows)

```

Et une connexion `psql` avec le compte dynamique fonctionne :

```

      current_user
-----
v-token-paylink--3oID1SPcRYEp3LHIYkhu-1783451429
(1 row)

```

Après révocation du lease, le rôle disparaît :

```
psql: error: ... FATAL:  role "v-token-paylink--3oID..." does not exist
```



Démo 4.1

Comptes PostgreSQL éphémères de bout en bout

`demos/module-04/demo-4-1-database-postgres.md`

3. Le cycle de vie des leases

Anatomie d'un lease

Un **lease** est le contrat de location d'un secret dynamique :

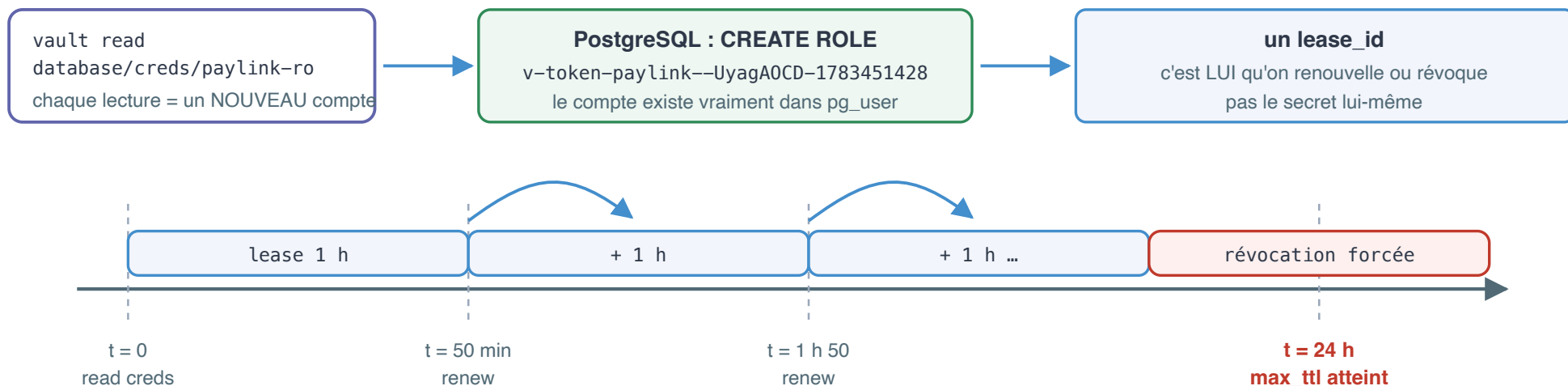
Champ	Rôle
<code>lease_id</code>	Identifiant unique – c'est lui qu'on renouvelle ou révoque, pas le secret
<code>lease_duration</code>	Durée restante avant expiration (repart de <code>default_ttl</code> à chaque renew)
<code>lease_renewable</code>	Le lease peut-il être prolongé ?

- Vault tient le registre de tous les leases actifs : `sys/leases/`
- À l'expiration ou à la révocation, Vault exécute les *revocation statements* (par défaut : `DROP ROLE`) – le compte disparaît de PostgreSQL
- Les tokens ont exactement le même mécanisme (module 1) : tout expire par défaut

Timeline d'un lease

Le lease : un bail sur un compte qui n'existait pas avant vous

default_ttl = 1 h — max_ttl = 24 h — chaque renew repart de default_ttl, sans jamais dépasser max_ttl



Qui renouvelle ?

L'application (SDK) · Vault Agent (module 5) · personne (job court : c'est très bien).
Piège classique : lire les creds au démarrage et ne jamais renouveler → panne à t + 1 h.

À l'échéance de max_ttl — ou si personne ne renouvelle

Vault exécute les revocation statements : DROP ROLE côté PostgreSQL.
psql: FATAL: role "v-token-paylink--..." does not exist

Renouveler, révoquer

```
# Prolonger un lease (repart pour default_ttl, plafonné par max_ttl)
v lease renew database/creds/paylink-readonly/5k0KHx9S0HJPVCYmSwitYKzJ
```

Key	Value
---	----
lease_id	database/creds/paylink-readonly/5k0KHx9S0HJPVCYmSwitYKzJ
lease_duration	1h
lease_renewable	true

```
# Révoquer immédiatement (compromission, décommission)
v lease revoke database/creds/paylink-readonly/5k0KHx9S0HJPVCYmSwitYKzJ

# Révoquer TOUT un sous-arbre (fin de TP, incident majeur)
v lease revoke -prefix database/creds/paylink-readonly
```

À l'expiration comme à la révocation : mêmes effets — Vault supprime le compte côté base. La différence est seulement *qui déclenche*.

Qui renouvelle ? L'app ou l'agent ?

Quelqu'un doit renouveler avant expiration – trois réponses possibles :

- **L'application elle-même** (SDK Vault) : boucle de renew intégrée, re-lecture des creds à l'approche de `max_ttl`, reconnexion du pool – puissant mais intrusif dans le code
- **Vault Agent** (module 5) : sidecar qui s'authentifie, lit les creds, les écrit dans un template, **renouvelle tout seul** et re-rend le fichier quand les creds changent – l'app reste ignorante de Vault
- **Personne** : acceptable pour un job batch court – le lease expire après le job, c'est même souhaitable

Piège classique : une app qui lit les creds au démarrage et ne les renouvelle jamais tombe en erreur `password authentication failed` après `default_ttl`.

Rotation du compte de gestion : rotate-root

Le compte de gestion (`postgres` / `rootpg`) est le dernier secret statique de la chaîne. Vault peut le faire tourner :

```
v write -f database/rotate-root/paylink-pg
```

- Vault change le mot de passe du compte de gestion **directement dans PostgreSQL** et ne le communique à personne
- Après rotation, **plus personne ne connaît ce mot de passe** — seul Vault peut l'utiliser

Précautions avant de lancer :

- Personne d'autre ne doit utiliser ce compte (supervision, scripts d'admin...)
- Prévoyez un compte de secours superuser côté PostgreSQL
- Irréversible : pas de « lecture » du nouveau mot de passe, par conception



Exercice 4.1

Cycle de vie d'un lease en accéléré

`exercices/module-04/exercice-4-1-cycle-de-vie-lease.md` — 25 min

4. Le moteur **transit** : chiffrement as a service

Pourquoi chiffrer via Vault ?

PayLink doit stocker des numéros de cartes chiffrés. L'approche naïve : une clé AES dans la config de l'app.

Problèmes : la clé traîne dans la config/le code, chaque service la réimplémente, rotation = redéploiement général, aucune trace de qui chiffre quoi.

Avec **transit**, la clé ne quitte jamais Vault :

- L'app envoie le clair, Vault renvoie le chiffré (et inversement) – API simple
- Rotation de clé **centralisée**, sans redéploiement
- Chaque opération est **auditée** et soumise aux **policies** (une app peut chiffrer sans pouvoir déchiffrer)
- Vault ne stocke **pas** les données : c'est du chiffrement *as a service*, pas du stockage

Créer une clé, chiffrer

```
v secrets enable transit
v write -f transit/keys/paylink-cards type=aes256-gcm96
```

Le plaintext s'envoie **en base64** (l'API transporte des octets arbitraires, pas du texte) :

```
v write transit/encrypt/paylink-cards \
  plaintext=$(printf '%s' '4970-1234-5678-9010' | base64)
```

Key	Value
---	-----
ciphertext	vault:v1:FQ9Dhk+8tFzHSZSHkMzboJrrJoBZJVzZhXQ0Ih4BQbgMekkntBKtJNUvAhwXbvI=
key_version	1

Format `vault:v1:...` : `v1` = **version de la clé** utilisée. Le ciphertext se stocke tel quel en base (colonne `TEXT`).

Déchiffrer

```
v write -field=plaintext transit/decrypt/paylink-cards \  
  ciphertext="vault:v1:FQ9Dhk+8tFzHSZSHkMzboJrrJoBZJVzZhxQ0Ih4BQbgMekkntBKtJNUvAhwXbvI=" \  
  | base64 -d
```

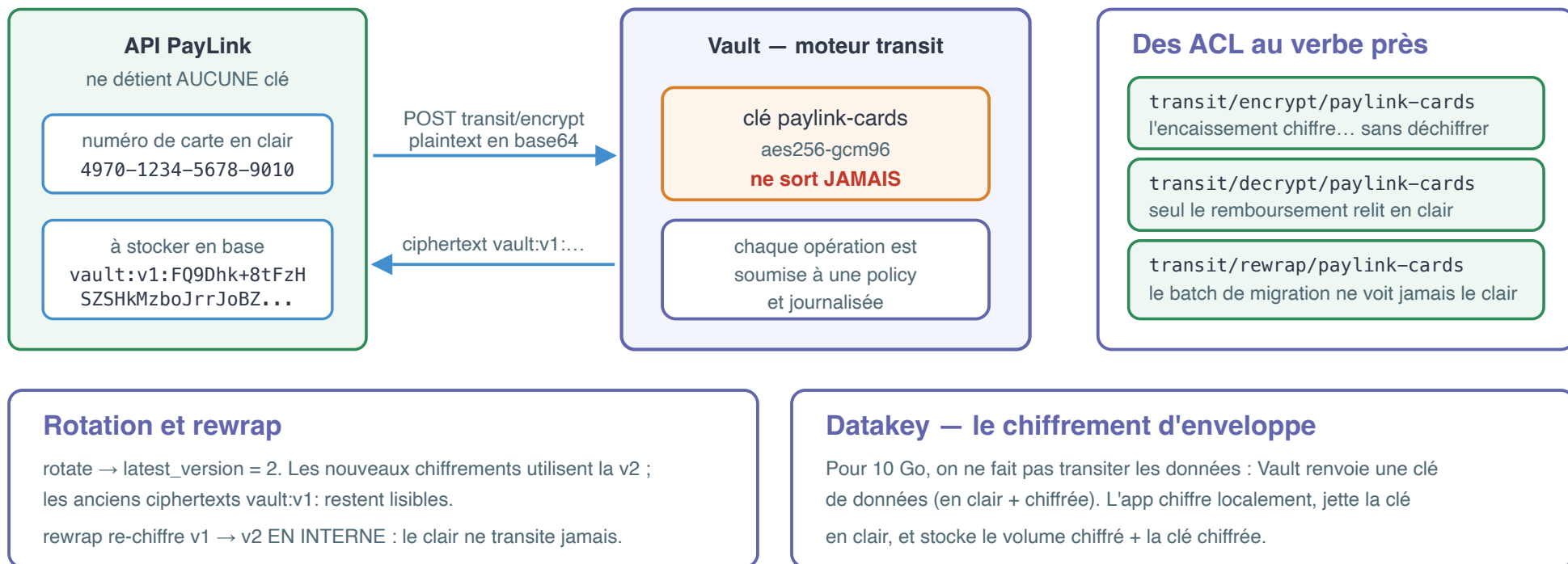
4970-1234-5678-9010

- Le résultat du decrypt est aussi en **base64** : ne pas oublier le `base64 -d` final
- Vault lit la version dans le préfixe `vault:vN:` et choisit la bonne clé tout seul

transit en une vue

transit — chiffrement as a service : la clé ne quitte jamais Vault

Vault ne stocke pas vos données : il chiffre et déchiffre à la demande, sous policy et sous audit



Rotation de clé

```
v write -f transit/keys/paylink-cards/rotate  
v read transit/keys/paylink-cards
```

```
keys                map[1:1783451454 2:1783451454]  
latest_version      2  
min_decryption_version 1
```

- Les **nouveaux** chiffrements utilisent la v2 (`latest_version`)
- Les **anciens** ciphertexts `vault:v1:` restent déchiffrables tant que `min_decryption_version` ≤ 1
- Pour interdire les vieux ciphertexts (après re-chiffrement du stock) :

```
v write transit/keys/paylink-cards/config min_decryption_version=2
```

Rotation possible en un clic, à la demande ou périodique (`auto_rotate_period`) – sans toucher aux applications.

Rewrap : re-chiffrer sans exposer le clair

Après rotation, comment migrer les données déjà chiffrées en v1 ?

```
v write transit/rewrap/paylink-cards \
  ciphertext="vault:v1:rli0NrFcQrJAhqYw3AEgjxkp56jQgAyQjuNuWNYL2X0EOPqN0nqD1m2pFWbcqkk="
```

Key	Value
---	-----
ciphertext	vault:v2:cT0S2AV+skh1lE/rFgXdsYhY86QcYZoXw99Mp0WhPGUV6cwCCSuMGFW167g1TTo=
key_version	2

- Vault déchiffre en v1 et re-chiffre en v2 **en interne** : le clair ne transite jamais vers le client
- Un batch de migration peut donc tourner avec une policy qui n'autorise **que** `rewrap` — il ne verra jamais un seul numéro de carte
- Ensuite seulement : remonter `min_decryption_version`

Datakey : le chiffrement d'enveloppe

Chiffrer 10 Go via l'API transit ? Non — on demande une **clé de données** :

```
v read transit/datakey/plaintext/paylink-cards
```

ciphertext	vault:v2:xmKl2hXjc7W5VovmI/a5LRqJnLGcBTBRsKT7SSrV91/mYtoo8alfZsXm9mYDffGdryA9eIS/6IG0fJVO
plaintext	cow+DrivxRem5FDv5RxGAlfh1MvmNFijxH/QjD4owZU=

1. datakey -> Vault renvoie la clé en clair (b64) + la même clé chiffrée
2. l'app chiffre le gros fichier LOCALEMENT avec la clé en clair (AES)
3. l'app JETTE la clé en clair, stocke : fichier chiffré + clé chiffrée
4. pour relire : transit/decrypt de la clé chiffrée -> clé en clair -> déchiffrement local

Le trafic vers Vault reste minuscule (une clé), le volume se chiffre en local. La clé maîtresse `paylink-cards` ne sort toujours pas de Vault.

ACL fines sur transit

Chaque opération transit est un **path** distinct → politiques au verbe près :

```
# L'API d'encaissement chiffre, mais ne déchiffre JAMAIS
path "transit/encrypt/paylink-cards" {
  capabilities = ["update"]
}

# Seul le service de remboursement déchiffre
path "transit/decrypt/paylink-cards" {
  capabilities = ["update"]
}
```

Le batch de migration ne voit jamais le clair

```
path "transit/rewrap/paylink-cards" {  
  capabilities = ["update"]  
}
```

- Une compromission du front d'encaissement ne permet **pas** de relire le stock de cartes : il n'a pas **decrypt**
- Le batch qui re-chiffre après rotation n'a que **rewrap** : le clair ne transite jamais jusqu'à lui
- À noter : **encrypt** / **decrypt** sont des **update** (POST), pas des **read**

Démo 4.2

Transit : encrypt, rotate, rewrap, datakey

`demos/module-04/demo-4-2-transit.md`

5. Choisir le bon moteur

KV statique vs database vs transit

	KV v2 (statique)	database (dynamique)	transit (chiffrement)
Vault stocke	le secret	rien (il le fabrique)	la clé, jamais les données
Le secret est	partagé, durable	unique par consommateur, jetable	il n'y a pas de « secret » applicatif
Rotation	manuelle / outillée	automatique (TTL)	rotation de clé + rewrap
Révocation	changer + redéployer	<code>lease revoke</code> , unitaire	<code>min_decryption_version</code>
Imputabilité	faible (compte partagé)	forte (un compte par lease)	forte (audit par opération)
Cas PayLink	clé d'API partenaire, DSN legacy	accès PostgreSQL de l'API	numéros de cartes en base
Limite	rotation à votre charge	la cible doit être pilotable par Vault	latence d'un appel réseau par op

Règle pratique : **statique si la cible l'impose, dynamique dès que la cible le permet, transit dès que la donnée est sensible.**

TP 4

PayLink : credentials dynamiques + chiffrement des cartes

`tp/module-04/tp-4-dynamique-et-chiffrement.md` — 50 min

Récap du module

- Un secret statique partagé = anonyme, éternel, irrévocable finement ; un secret **dynamique** = un compte par consommateur, TTL, révocable, audité
- Moteur `database` : `config/` (connexion de gestion), `roles/` (creation_statements SQL), `creds/` (consommation) — les comptes existent réellement dans `pg_user` et disparaissent au revoke
- **Lease** : `lease_id`, `lease_duration`, `renew` avant expiration (plafonné par `max_ttl`), `revoke` à la demande — le renouvellement revient à l'app ou au Vault Agent
- `rotate-root` : Vault s'approprie le compte de gestion — irréversible, à préparer
- `transit` : la clé ne sort jamais de Vault ; plaintext en **base64** ; format `vault:vN:` ; rotation + `rewrap` sans exposer le clair ; `datakey` pour les gros volumes ; ACL au verbe près (encrypt sans decrypt)
- Choix : statique si imposé, dynamique si possible, transit si donnée sensible

Cheatsheet : `cheatsheet/module-04.md`

Quiz – 1/5

Q1. Citez trois inconvénients d'un compte PostgreSQL statique partagé par tous les services, que les secrets dynamiques corrigent.

Q2. Dans `database/config/paylink-pg`, à quoi servent les gabarits `{{username}}` et `{{password}}` dans la `connection_url`, et quel compte remplacent-ils ?

Quiz – 2/5

Q3. Que se passe-t-il exactement, côté Vault et côté PostgreSQL, quand une application exécute `vault read database/creds/paylink-readonly` ?

Q4. À quoi sert la clause `VALID UNTIL '{{expiration}}'` dans les `creation_statements`, alors que Vault révoque déjà le compte à l'expiration du lease ?

Quiz – 3/5

Q5. Un lease a `default_ttl=1h` et `max_ttl=24h`. L'application le renouvelle toutes les 50 minutes. Que se passe-t-il au bout de 24 h ?

Q6. Après `vault write -f database/rotate-root/paylink-pg`, comment récupérez-vous le nouveau mot de passe du compte de gestion ? Quelle précaution fallait-il prendre avant ?

Quiz – 4/5

- Q7.** Pourquoi le `plaintext` envoyé à `transit/encrypt/...` doit-il être encodé en base64 ?
- Q8.** Que signifie le préfixe `vault:v1:` d'un ciphertext, et que se passe-t-il si on tente de le déchiffrer après avoir positionné `min_decryption_version=2` ?

Quiz – 5/5

- Q9.** Quelle est la différence entre `transit/rewrap` et une séquence decrypt puis encrypt côté client ? Quel avantage de sécurité pour un batch de migration ?
- Q10.** L'API d'encaissement PayLink doit pouvoir chiffrer les cartes mais jamais les relire en clair. Quel(s) path(s) et quelle(s) capability(ies) sa policy doit-elle contenir ?

Questions ?

Module suivant : Vault dans Kubernetes

Module 5

Vault dans Kubernetes

Vault Agent · Agent Injector · Vault Secrets Operator · CSI

Objectifs du module

À l'issue de ce module, vous serez capable de :

- Expliquer le rôle de **Vault Agent** : auto-auth, sinks, templates
- Configurer la **méthode d'auth kubernetes** (TokenReview, rôles liés aux ServiceAccounts)
- Injecter des secrets dans un pod avec l'**Agent Injector** (annotations, init vs sidecar)
- Synchroniser des secrets Vault vers des **Secrets Kubernetes natifs** avec le **Vault Secrets Operator**
- Positionner le **CSI provider** et **choisir** le bon mécanisme selon le cas d'usage

Fil rouge : **MediTrack** déployé sur un cluster kind.

Plan

1. **Vault Agent** — la brique de base, hors Kubernetes d'abord
2. **Auth kubernetes** — l'identité du pod, en pratique
3. **Agent Injector** — le sidecar automatique par annotations
4. **Vault Secrets Operator** — des Secrets K8s natifs pilotés par CRD
5. **CSI provider et comparaison** — choisir son intégration

Démos 5.1, 5.2, 5.3 — Exercice 5.1 — TP 5 — Quiz

1. Vault Agent

**Comprendre la brique avant de la voir dans
Kubernetes**

Pourquoi Vault Agent ?

Jusqu'ici, c'est **votre code** qui fait tout :

- s'authentifier (AppRole), stocker le token, le **renouveler** avant expiration
- relire les secrets, gérer les erreurs réseau, les retries...

Vault Agent est un **démon** (le même binaire `vault`) qui prend tout cela en charge **à côté** de l'application :

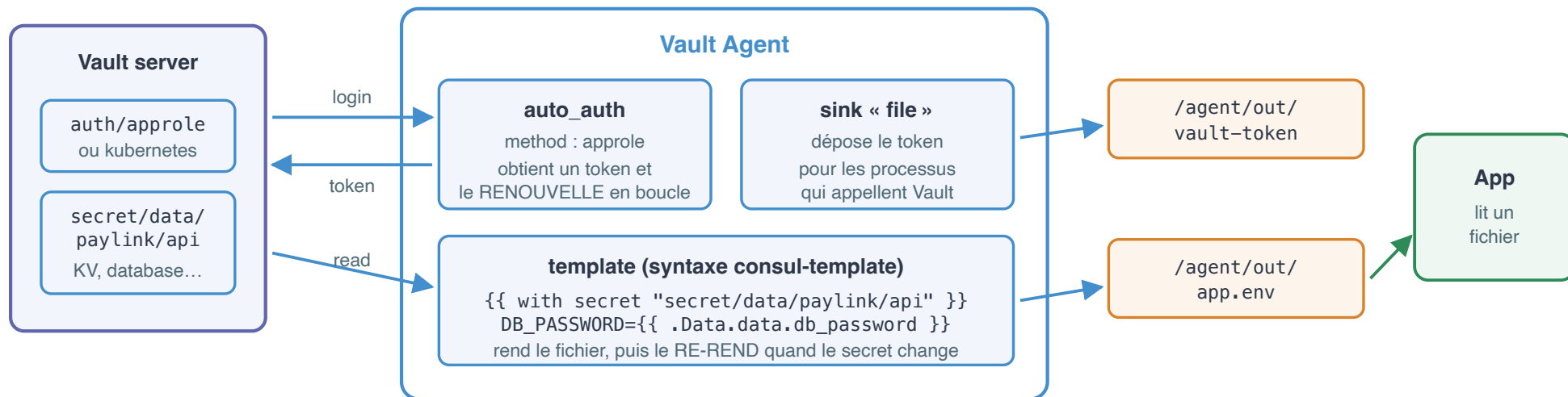
Fonction	Ce que fait l'agent
Auto-auth	S'authentifie (AppRole, kubernetes, ...) et renouvelle le token en boucle
Sink	Dépose le token dans un fichier pour d'autres processus
Template	Rend des fichiers de config à partir des secrets et les re-rend en cas de changement
Cache / proxy	Peut servir de proxy API local qui injecte le token

L'application **ne parle plus à Vault** : elle lit un fichier. Zéro SDK, zéro token dans le code.

Architecture de Vault Agent

Vault Agent — l'application ne parle plus à Vault : elle lit un fichier

le même binaire vault, lancé en démon à côté de l'app : il s'authentifie, renouvelle et rend les fichiers



Ce que l'agent supprime de votre code

le login, le stockage du token, la boucle de renew, les retries réseau, la relecture des secrets. Zéro SDK, zéro token en dur.

Piège testé en lab : les secrets KV statiques n'ont pas de lease

L'agent les relit périodiquement — toutes les 5 minutes PAR DÉFAUT.
`template_config { static_secret_render_interval = "10s" }`

auto_auth : method + sink

```
auto_auth {  
  method "approle" {  
    config = {  
      role_id_file_path      = "/agent/role_id"  
      secret_id_file_path    = "/agent/secret_id"  
      remove_secret_id_file_after_reading = false  
    }  
  }  
  
  sink "file" {  
    config = {  
      path = "/agent/out/vault-token"  
    }  
  }  
}
```

auto_auth : les paramètres à connaître

- **method** : comment l'agent obtient son token — **aprole** ici, **kubernetes** dans un pod
- **role_id_file_path** / **secret_id_file_path** : les identifiants AppRole livrés **en fichiers** (module 2)
- **remove_secret_id_file_after_reading** : **true** par défaut — l'agent **supprime** le secret_id après lecture (usage unique) ; **false** en lab pour pouvoir redémarrer le conteneur
- **sink "file"** : le token est déposé pour les processus qui veulent appeler Vault eux-mêmes

Templates : déclarer le rendu

```
template {  
  source      = "/agent/app.env.ctmpl"  
  destination = "/agent/out/app.env"  
}
```

`app.env.ctmpl` — syntaxe **consul-template** :

```
# Genere par Vault Agent - ne pas editor  
{{ with secret "secret/data/paylink/api" -}}  
DB_USER={{ .Data.data.db_user }}  
DB_PASSWORD={{ .Data.data.db_password }}  
API_KEY={{ .Data.data.api_key }}  
{{- end }}
```


Lire la syntaxe consul-template

- `secret "secret/data/..."` : chemin **API** — avec `data/` pour KV v2, comme dans les politiques du module 3
- `.Data.data.xxx` : deux niveaux — l'enveloppe API (`.Data`), puis les données KV v2 (`.data`)
- `-` dans `{{ ... - }}` : supprime les sauts de ligne parasites
- L'agent rend le fichier au démarrage, puis le **re-rend** quand le secret change

agent.hcl (1/2) – serveur et authentification

```
pid_file = "/agent/pidfile"

vault {
  address = "http://vault:8200"
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path      = "/agent/role_id"
      secret_id_file_path    = "/agent/secret_id"
      remove_secret_id_file_after_reading = false
    }
  }
}

sink "file" {
  config = { path = "/agent/out/vault-token" }
}
}
```

Où joindre Vault, comment s'y authentifier, où déposer le token.

agent.hcl (2/2) – rendu des templates

```
template_config {  
    static_secret_render_interval = "10s"  
}  
  
template {  
    source      = "/agent/app.env.ctmpl"  
    destination = "/agent/out/app.env"  
}
```

- `template_config` : la fréquence de re-lecture des secrets **statiques** (5 min par défaut – voir la slide suivante)
- `template` : le gabarit à rendre, et le fichier que lira l'application

Un seul conteneur : `vault agent -config=/agent/agent.hcl` – même image `hashicorp/vault:1.20`.

Piège testé : le re-rendu des secrets statiques

Les secrets **KV statiques n'ont pas de lease** : l'agent ne peut pas être notifié d'un changement. Il **re-lit périodiquement** le secret... toutes les **5 minutes par défaut**.

Constat en lab : on fait tourner le mot de passe côté Vault, et une minute plus tard :

```
# Genere par Vault Agent - ne pas editer
DB_USER=paylink_app
DB_PASSWORD=Rot4ted!2026      <-- toujours l'ancienne valeur !
API_KEY=pk_live_99Z
```

La solution pour raccourcir la fenêtre (démonstration : 10 s) :

```
template_config {
  static_secret_render_interval = "10s"
}
```

- En production : gardez un intervalle raisonnable (chaque re-rendu = une lecture Vault)
- Les secrets **dynamiques** (module 4), eux, sont re-rendus pilotés par leur **lease**



Démo 5.1

Vault Agent standalone sous Podman

Auto-auth AppRole → sink → template → rotation

```
demos/module-05/demo-5-1-vault-agent.md
```

2. Auth kubernetes

L'identité du pod, sans secret à distribuer

Le principe : le ServiceAccount comme identité

Dans Kubernetes, chaque pod porte déjà une identité vérifiable : le **token de son ServiceAccount**, monté par la plateforme dans :

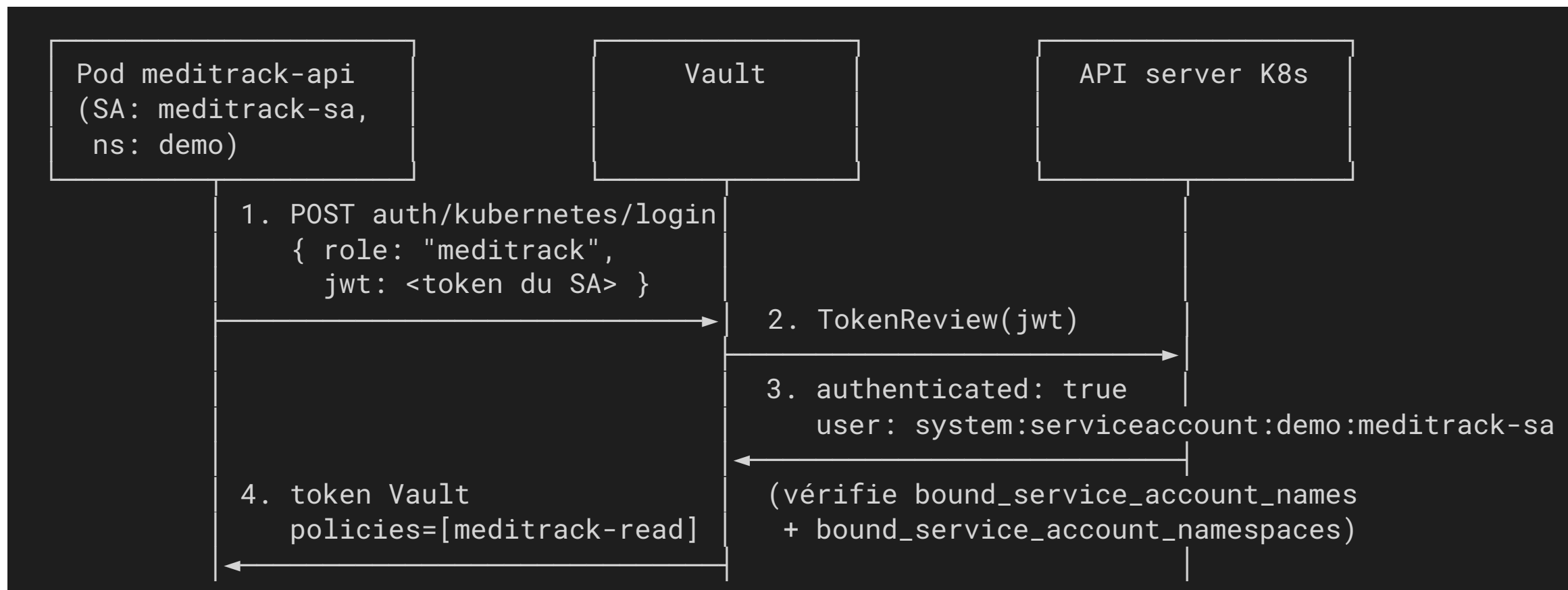
```
/var/run/secrets/kubernetes.io/serviceaccount/token
```

L'auth `kubernetes` de Vault exploite ce token :

1. Le pod présente son token de ServiceAccount à Vault (`auth/kubernetes/login`)
2. Vault demande à l'**API server Kubernetes** de le valider (**TokenReview**)
3. Si le couple **ServiceAccount + namespace** correspond à un **rôle** → token Vault avec les politiques du rôle

C'est la réponse au « secret zéro » du module 2 : **rien à livrer au pod**, la plateforme fournit l'identité. Même modèle que l'auth AWS/Azure : *la plateforme atteste, Vault vérifie.*

Flux TokenReview



Configuration côté Vault (testée en lab)

Trois commandes, exécutées **dans** le cluster (`kubectl -n vault exec vault-0`):

```
vault auth enable kubernetes
# Success! Enabled kubernetes auth method at: kubernetes/

vault write auth/kubernetes/config \
  kubernetes_host="https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_SERVICE_PORT"
# Success! Data written to: auth/kubernetes/config

vault write auth/kubernetes/role/meditrack \
  bound_service_account_names=meditrack-sa \
  bound_service_account_namespaces=demo \
  policies=meditrack-read \
  ttl=1h
```

Ce que fait chacune des trois commandes

- **kubernetes_host** : l'URL de l'API server — depuis le pod Vault, les **variables d'environnement injectées par K8s** suffisent
- **Le rôle est le pivot** : il lie **(ServiceAccount, namespace)** → **policies**.
C'est lui qui traduit une identité Kubernetes en droits Vault
- Vault (dans le cluster) utilise **son propre token de SA** pour appeler TokenReview : aucun credential à fournir

Le warning « audience » — observé en lab

À la création du rôle, Vault répond :

```
WARNING! The following warnings were returned from Vault:
```

```
* Role meditrack does not have an audience configured. While audiences are not required, consider specifying one if your use case would benefit from additional JWT claim verification.
```

Pourquoi ce warning, et que faire

Un token de ServiceAccount porte une claim `aud` (audience = destinataire prévu). Sans `audience` sur le rôle, Vault accepte le token **quel que soit son destinataire** → un token projeté pour *un autre service* pourrait être rejoué contre Vault.

En production :

```
vault write auth/kubernetes/role/meditrack audience="vault" ...
```

...et côté pod, un token **projeté** avec la même audience (`serviceAccountToken.audience`).

En lab dev, le warning est acceptable — mais sachez l'expliquer.

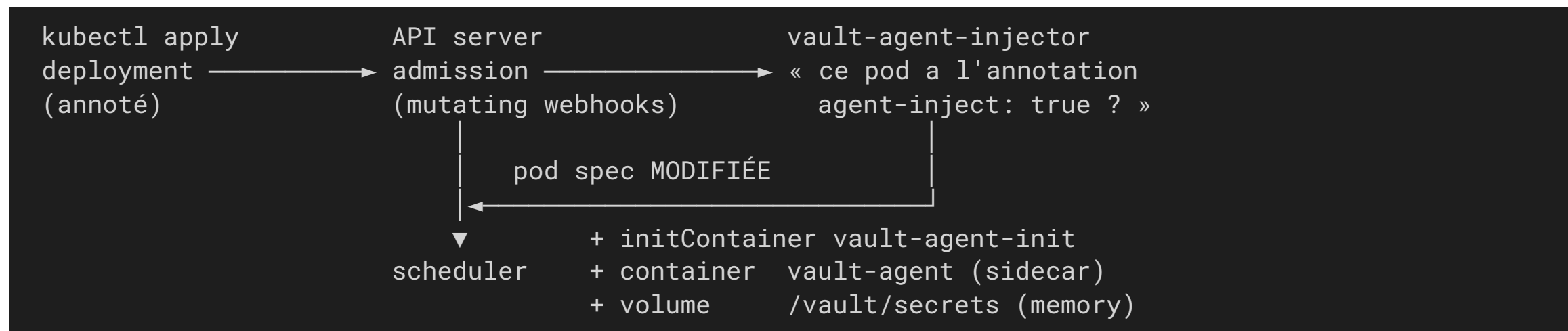
Bonnes pratiques : jamais `bound_service_account_names=*`, un rôle **par service**, des TTL courts.

3. Vault Agent Injector

Le sidecar automatique, piloté par annotations

Le mutating webhook

Le chart Helm `hashicorp/vault` installe (avec `injector.enabled=true`) un déploiement `vault-agent-injector` enregistré comme **MutatingWebhookConfiguration** :



- Le webhook **réécrit la spec du pod à la création** : ni l'image, ni le code, ni le manifest du conteneur applicatif ne changent
- Le vault-agent injecté fait de l'**auto-auth** `kubernetes` (section 2) et rend ses templates dans un volume **en mémoire** partagé : `/vault/secrets/`

Les annotations essentielles

Tout se pilote par annotations sur le **template de pod** (pas sur le Deployment lui-même) :

Annotation	Rôle
<code>vault.hashicorp.com/agent-inject: "true"</code>	Active l'injection pour ce pod
<code>vault.hashicorp.com/role: "meditrack"</code>	Rôle d'auth kubernetes utilisé par l'agent
<code>vault.hashicorp.com/agent-inject-secret-<nom></code>	Secret à rendre dans <code>/vault/secrets/<nom></code>
<code>vault.hashicorp.com/agent-inject-template-<nom></code>	Template consul-template pour ce fichier
<code>vault.hashicorp.com/agent-pre-populate-only: "true"</code>	Init container seul, pas de sidecar

Le minimum vital

```
annotations:  
  vault.hashicorp.com/agent-inject: "true"  
  vault.hashicorp.com/role: "meditrack"  
  vault.hashicorp.com/agent-inject-secret-app.env: "secret/data/meditrack/api"
```

Trois annotations suffisent à déclencher l'injection.

Sans template, le fichier rendu contient le **struct Go brut** du secret — en pratique on fournit presque toujours un template.

Le template d'injection (testé en lab)

```
template:
  metadata:
    labels:
      app: meditrack-api
    annotations:
      vault.hashicorp.com/agent-inject: "true"
      vault.hashicorp.com/role: "meditrack"
      vault.hashicorp.com/agent-inject-secret-app.env: "secret/data/meditrack/api"
      vault.hashicorp.com/agent-inject-template-app.env: |
        {{ with secret "secret/data/meditrack/api" -}}
        DB_USER={{ .Data.data.db_user }}
        DB_PASSWORD={{ .Data.data.db_password }}
        {{- end }}
  spec:
    serviceAccountName: meditrack-sa
```

Le template d'injection – les trois points clés

- Même syntaxe consul-template que Vault Agent standalone – **c'est** un Vault Agent, injecté automatiquement
- Le `<nom>` du fichier vient du **suffixe** de l'annotation :
`...-secret-app.env` → `/vault/secrets/app.env`
- `serviceAccountName` doit correspondre au `bound_service_account_names` du rôle Vault, sinon le login est refusé

Init vs sidecar

Par défaut, le webhook injecte **deux** conteneurs agent :

Conteneur	Moment	Rôle
<code>vault-agent-init</code>	initContainer	Rend les secrets avant le démarrage de l'app
<code>vault-agent</code>	sidecar	Reste en vie : renouvelle le token, re-rend en cas de rotation

```
# Secrets consommés une seule fois au démarrage ? Init seul :
vault.hashicorp.com/agent-pre-populate-only: "true"
```

- **Init seul** : jobs, migrations, apps qui lisent leur config une fois — pod plus léger, pas de conteneur permanent
- **Sidecar** : services longue durée qui doivent suivre les rotations — mais l'app doit **relire le fichier** (watch, reload, ou `agent-inject-command` pour signaler le process)
- Pod visible en `2/2 Running` : conteneur `app` + conteneur `vault-agent`

Ce que voit l'application

Vérifié en lab (cluster kind, K8s 1.35) :

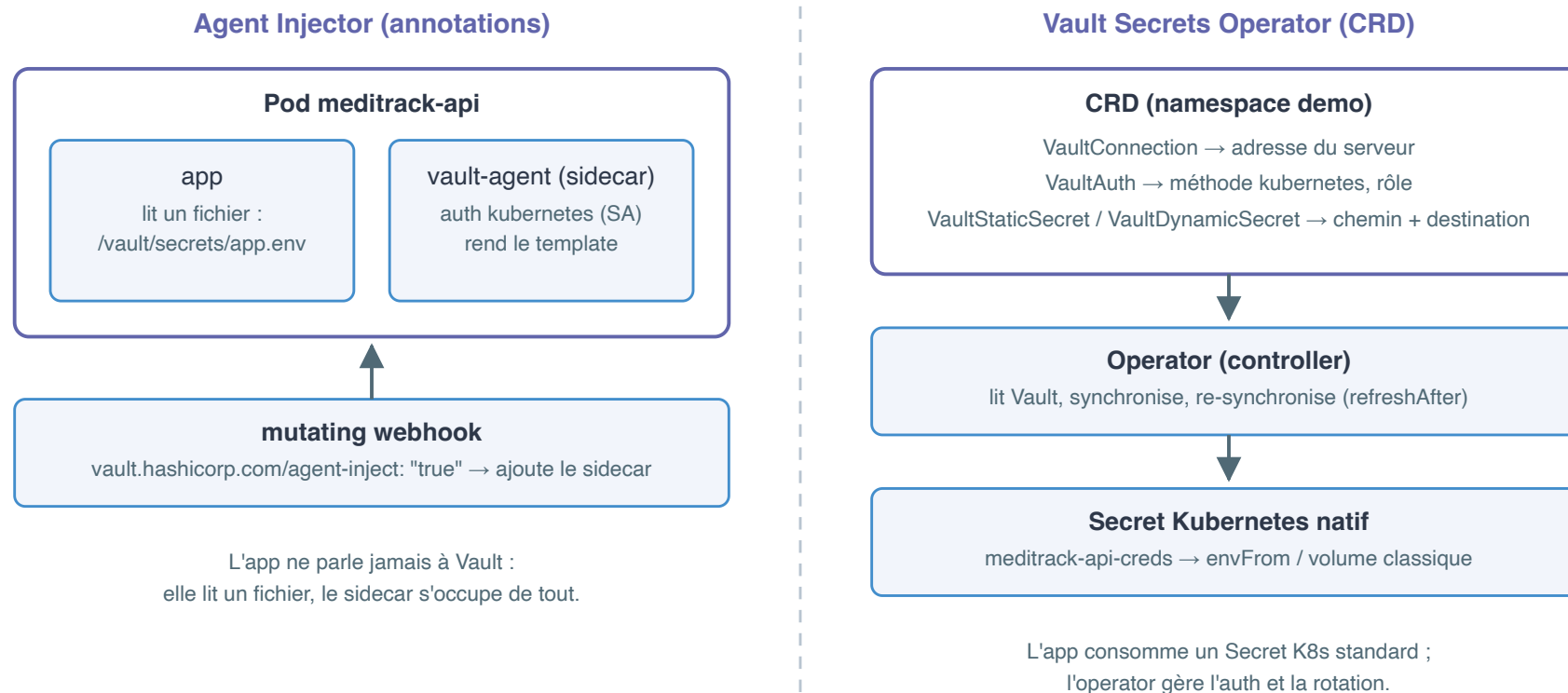
```
kubectl -n demo get pod -l app=meditrack-api \
  -o jsonpath='{.items[0].spec.containers[*].name}'
# app vault-agent

kubectl -n demo exec deploy/meditrack-api -c app -- cat /vault/secrets/app.env
# DB_USER=meditrack
# DB_PASSWORD=K8sInject!2026
```

Points clés :

- Le secret n'existe **nulle part dans etcd** : volume `emptyDir` **en mémoire**, local au pod
- Le conteneur `app` n'a **aucune** dépendance à Vault : il lit un fichier
- Le token Vault du sidecar vit dans le pod, jamais dans un Secret K8s
- Contrepartie : l'app doit savoir lire un **fichier** (pas de `envFrom` possible)

Injector et VSO côte à côte



Démo 5.2

Injection dans MediTrack sur kind

Auth kubernetes → annotations → pod 2/2 → `/vault/secrets/app.env`

`demos/module-05/demo-5-2-injector.md`



Exercice 5.1

Annoter un deployment pour l'injection

~25 min — `exercices/module-05/exercice-5-1-annotations.md`

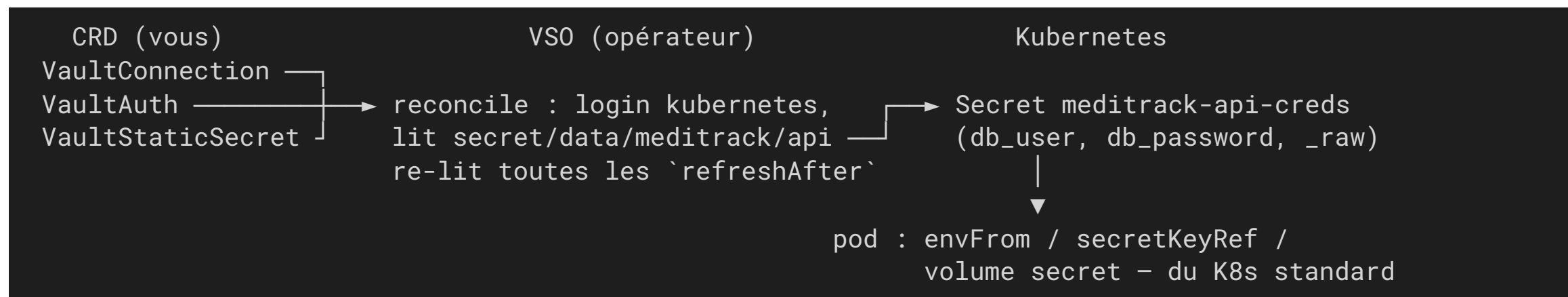
À partir d'un deployment MediTrack fourni **sans** annotations, injectez `secret/data/meditrack/api` au format properties Java.

4. Vault Secrets Operator

Des Secrets Kubernetes natifs, pilotés par CRD

Le principe : un opérateur, des CRD, des Secrets natifs

Approche inverse de l'Injector : au lieu d'amener un agent **dans chaque pod**, un **opérateur central** (chart `hashicorp/vault-secrets-operator`) synchronise Vault → **Secrets Kubernetes natifs**.



- L'application consomme un **Secret K8s ordinaire** : `envFrom`, `secretKeyRef`, volume
- Aucun sidecar, aucune annotation, aucun changement dans les pods
- Contrepartie : le secret **vit dans etcd** (chiffrement etcd + RBAC à soigner)

VaultConnection — où est Vault

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultConnection
metadata:
  name: vault-connection
  namespace: demo
spec:
  address: http://vault.vault.svc:8200
```

VaultConnection déclare **où** joindre Vault : adresse du service, TLS éventuel. Une seule ressource sert ensuite à toutes les CRD du namespace.

VaultAuth – comment s'authentifier

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultAuth
metadata:
  name: vault-auth
  namespace: demo
spec:
  vaultConnectionRef: vault-connection
  method: kubernetes
  mount: kubernetes
  kubernetes:
    role: vso-demo
    serviceAccount: default
```

- **VaultAuth** : la **méthode** (**kubernetes**), le mount, le rôle Vault et le ServiceAccount utilisé pour le login
- Le rôle Vault **vso-demo** doit lier ce SA/namespace aux bonnes politiques – même mécanique qu'en section 2

VaultStaticSecret

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata:
  name: meditrack-api
  namespace: demo
spec:
  vaultAuthRef: vault-auth
  mount: secret
  type: kv-v2
  path: meditrack/api
  refreshAfter: 15s
  destination:
    name: meditrack-api-creds
    create: true
```

VaultStaticSecret – les champs qui comptent

Champ	Sens
<code>mount</code> / <code>type</code> / <code>path</code>	Moteur KV v2 <code>secret/</code> , chemin logique <code>meditrack/api</code> (sans <code>data/</code> : VSO le gère)
<code>refreshAfter</code>	Période de re-lecture côté Vault (15 s en démo ; minutes en prod)
<code>destination.name</code>	Nom du Secret K8s créé/synchronisé
<code>destination.create</code>	VSO crée le Secret s'il n'existe pas

Le `path` est **logique** : contrairement aux policies (module 3), on n'écrit pas `data/` – l'opérateur l'ajoute lui-même.

Rotation et resynchronisation (testées en lab)

Le Secret K8s créé par VSO :

```
kubectl -n demo get secret meditrack-api-creds \
  -o go-template='{{ range $k, $v := .data }}{{ $k }} = {{ $v | base64decode }}{{ "\n" }}{{ end }}'
# _raw = {"data":{"db_password":"K8sInject!2026","db_user":"meditrack"}, ...}
# db_password = K8sInject!2026
# db_user = meditrack
```

La resynchronisation, mesurée en lab

Rotation côté Vault → **resync automatique** en moins de 25 s
(avec `refreshAfter: 15s`):

```
vault kv patch secret/meditrack/api db_password='RotatedByVS0!99' # version 2
# ... < 25 s plus tard :
kubectl -n demo get secret meditrack-api-creds \
  -o jsonpath='{.data.db_password}' | base64 -d
# RotatedByVS0!99
```

Attention : `envFrom` fige les valeurs **au démarrage du conteneur**.

Pour que les pods suivent la rotation, ajoutez `spec.rolloutRestartTargets` dans la CRD – VSO déclenche alors un rollout à chaque resync.

VaultDynamicSecret

Même logique pour les **secrets dynamiques** (module 4) – ici le moteur `database` :

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultDynamicSecret
metadata:
  name: meditrack-db
  namespace: demo
spec:
  vaultAuthRef: vault-auth
  mount: database
  path: creds/meditrack-role
  destination:
    name: meditrack-db-creds
    create: true
  rolloutRestartTargets:
    - kind: Deployment
      name: meditrack-api
```


VaultDynamicSecret — ce qui change

- **path: creds/<rôle>** : chaque synchronisation obtient un **lease** ; VSO **renouvelle** puis régénère à l'expiration
- **Pas de refreshAfter** : c'est le **TTL du lease** qui pilote le cycle, pas une période fixe
- **rolloutRestartTargets** : indispensable ici — les anciens credentials sont **révoqués** côté base, les pods doivent redémarrer avec les nouveaux sous peine de **password authentication failed**



Démo 5.3

Vault Secrets Operator

CRD → Secret K8s natif → rotation → resync < 25 s

```
demos/module-05/demo-5-3-vso.md
```

5. CSI provider et comparaison

Choisir le bon mécanisme

Le CSI provider en une slide

Troisième voie : le **Vault CSI provider**, branché sur le **Secrets Store CSI Driver** (projet Kubernetes générique, aussi utilisé pour AWS/Azure/GCP) :

```

pod → volume CSI (secrets-store.csi.k8s.io) → CSI driver → Vault provider → Vault
      |
      └→ secrets montés en fichiers dans le pod (tmpfs)
  
```

- Le secret est **monté en volume** au démarrage du pod, via une CRD `SecretProviderClass`
- Comme l'Injector : **pas de Secret dans etcd** (sauf option `secretObjects` de sync explicite)
- Contrairement à l'Injector : pas de sidecar par pod, mais un **DaemonSet** (driver) sur chaque nœud
- Rotation possible mais optionnelle et périodique (fonctionnalité du driver à activer)

Positionnement : pertinent si le CSI driver est **déjà en place** pour d'autres providers cloud, ou si la politique interdit sidecars **et** Secrets etcd. Sinon, Injector ou VSO couvrent l'essentiel.

Injector vs VSO vs CSI

Critère	Agent Injector	Vault Secrets Operator	CSI provider
Où vit le secret	Fichier dans le pod (volume mémoire <code>/vault/secrets/</code>)	Secret K8s natif (etcd)	Fichier dans le pod (volume CSI tmpfs)
Consommation	Fichier uniquement	<code>envFrom</code> , <code>secretKeyRef</code> , volume	Fichier (+ sync Secret optionnelle)
Rotation	Sidecar re-rend en continu	Resync <code>refreshAfter</code> / lease + <code>rolloutRestartTargets</code>	Périodique, optionnelle
Dépendances	Webhook + 1-2 conteneurs par pod	1 opérateur central	CSI driver (DaemonSet) + provider par nœud
Modif. des manifests	Annotations sur le pod	Aucune côté pod (CRD à part)	Volume + SecretProviderClass
Secrets dynamiques	Oui (templates + leases)	Oui (<code>VaultDynamicSecret</code>)	Limité (pas de renew)
Cas d'usage type	Fichier de config, zéro secret etcd	Apps standard <code>envFrom</code> , GitOps, Helm charts tiers	CSI déjà standardisé multi-cloud

Recommandations

- **Vault Secrets Operator** — le choix par défaut aujourd'hui si les Secrets K8s natifs sont acceptables :
apps qui consomment `envFrom` / `secretKeyRef`, charts Helm tiers non modifiables, workflow GitOps (les CRD sont déclaratives). Soignez alors etcd (chiffrement at rest) et le RBAC sur les Secrets.
- **Agent Injector** — quand l'app veut un **fichier** de config et que la politique impose **zéro secret dans etcd** ; ou pour des templates riches (plusieurs secrets combinés dans un même fichier).
- **CSI provider** — si le Secrets Store CSI Driver est déjà votre standard multi-provider.
- Dans tous les cas : la sécurité repose sur la **section 2** — des rôles d'auth kubernetes précis (`bound_service_account_names` + `bound_service_account_namespaces` + `audience`), une policy par service, moindre privilège.

Pour MediTrack : injection pour comprendre, **VSO pour exploiter** — c'est le parcours du TP 5.

 TP 5

MediTrack sur kind : injection, puis VSO

~55 min — `tp/module-05/tp-5-meditrack-sur-kind.md`

Cluster kind + Helm → auth kubernetes → injection → migration VSO → preuve de rotation

Récap du module

- **Vault Agent** : auto-auth (method + sink) + templates consul-template — l'app ne parle plus à Vault ; secrets KV statiques re-rendus toutes les **5 min** par défaut (`static_secret_render_interval`)
- **Auth kubernetes** : le token du ServiceAccount validé par **TokenReview** ; le rôle lie `(SA, namespace)` → politiques ; configurez l'**audience** en production
- **Agent Injector** : mutating webhook + annotations `vault.hashicorp.com/*` → init + sidecar, secrets dans `/vault/secrets/`, jamais dans etcd
- **VSO** : `VaultConnection` / `VaultAuth` / `VaultStaticSecret` / `VaultDynamicSecret` → **Secrets K8s natifs**, resync automatique, `rolloutRestartTargets` pour propager
- **CSI** : volume monté sans Secret etcd, via le driver générique
- **Choix** : VSO si Secrets natifs acceptés ; Injector si fichier + zéro etcd ; CSI si déjà standardisé

Quiz – questions 1 et 2

Question 1 – Dans Vault Agent, quel composant dépose le token obtenu par l'auto-auth dans un fichier lisible par d'autres processus ?

- A. Le template
- B. Le sink
- C. Le cache
- D. La method

Question 2 – Par défaut, à quelle fréquence Vault Agent re-rend-il un template qui référence un secret KV statique ?

- A. En temps réel, dès que le secret change
- B. Toutes les 10 secondes
- C. Toutes les 5 minutes
- D. Jamais : uniquement au démarrage

Quiz – questions 3 et 4

Question 3 – Lors d'un login `auth/kubernetes/login`, comment Vault vérifie-t-il le token de ServiceAccount présenté par le pod ?

- A. Il le déchiffre avec la clé privée du cluster
- B. Il appelle l'API TokenReview de l'API server Kubernetes
- C. Il le compare à une liste de tokens stockée dans KV
- D. Il fait confiance au réseau du cluster

Question 4 – Que lie exactement un rôle de la méthode d'auth kubernetes ?

- A. Un namespace à un moteur de secrets
- B. Un pod à un token racine
- C. Un couple ServiceAccount + namespace à des policies
- D. Un Deployment à un Secret K8s

Quiz – questions 5 et 6

Question 5 – Quel couple minimal d'annotations faut-il pour que l'Injector injecte un agent dans un pod et qu'il puisse s'authentifier ?

- A. `agent-inject: "true"` et `role`
- B. `agent-inject-secret-...` seul
- C. `role` et `agent-inject-template-...`
- D. `agent-pre-populate-only: "true"` seul

Question 6 – Où l'application trouve-t-elle par défaut le fichier rendu par l'annotation `vault.hashicorp.com/agent-inject-secret-app.env` ?

- A. `/etc/vault/app.env`
- B. `/vault/secrets/app.env`
- C. Dans un Secret Kubernetes `app.env`
- D. Dans la variable d'environnement `APP_ENV`

Quiz – questions 7 et 8

Question 7 – Quelle annotation faut-il pour n'avoir qu'un init container (pas de sidecar permanent) ?

- A. `vault.hashicorp.com/agent-init-only: "true"`
- B. `vault.hashicorp.com/agent-pre-populate-only: "true"`
- C. `vault.hashicorp.com/agent-inject: "init"`
- D. `vault.hashicorp.com/sidecar: "false"`

Question 8 – Dans le Vault Secrets Operator, quelle CRD porte la méthode d'authentification et le rôle Vault utilisés pour la synchronisation ?

- A. VaultConnection
- B. VaultStaticSecret
- C. VaultAuth
- D. SecretProviderClass

Quiz – questions 9 et 10

Question 9 – Avec une `VaultStaticSecret` dont `destination.create: true`, où atterrit le secret synchronisé ?

- A. Dans un fichier `/vault/secrets/` du pod
- B. Dans un Secret Kubernetes natif portant le nom de `destination.name`
- C. Dans une ConfigMap
- D. Dans les logs de l'opérateur

Question 10 – Votre équipe déploie un chart Helm tiers qui attend un Secret K8s consommé en `envFrom`, et veut que la rotation soit propagée. Quel mécanisme choisir ?

- A. Agent Injector avec sidecar
- B. CSI provider sans sync
- C. VSO avec `VaultStaticSecret` + `rolloutRestartTargets`
- D. Un initContainer maison qui appelle l'API Vault

Fin du module 5

Prochain module : Vault dans le pipeline CI/CD

Auth JWT/OIDC, patterns d'intégration, anti-patterns

Module 6

Vault dans le pipeline CI/CD et bonnes pratiques

Auth JWT/OIDC · bound claims · patterns d'intégration · synthèse

Objectifs du module

À l'issue de ce module, vous serez capable de :

- Identifier les **anti-patterns** d'intégration Vault dans un pipeline CI/CD
- Expliquer le flux **JWT/OIDC** : id_token signé par la plateforme, vérifié par Vault
- Configurer un rôle `jwt` avec **bound_audiences** et **bound_claims** (portée par projet et par branche)
- Faire tourner un job de CI **sans aucun secret stocké** (fil rouge : le déploiement PayLink)
- **Choisir** le bon pattern d'intégration Vault selon le contexte — synthèse de la formation

Durée : 2 h — dernier module, clôture de la formation.

Plan

1. **Anti-patterns CI/CD** — ce qu'on trouve (trop) souvent dans les pipelines
2. **Auth JWT/OIDC** — le pipeline prouve son identité
3. **Pipelines réels** — GitLab CI et GitHub Actions en pratique
4. **Choisir son intégration** — synthèse des patterns de la formation

Démo 6.1 — Exercice 6.1 — TP 6 — Récap de la formation — Quiz — Plan d'action

1. Anti-patterns CI/CD

Ce que l'audit de PayLink a trouvé dans le pipeline

L'anti-pattern n° 1 : **VAULT_TOKEN** en variable de CI

```
GitLab > Settings > CI/CD > Variables
```

```
VAULT_TOKEN = hvs.CAESIJ... [Protected] [Masked] créé il y a 11 mois
```

C'est exactement le « mot de passe en dur » que Vault devait éliminer — simplement déplacé dans la CI.

Pourquoi, même « Protected » et « Masked »

- **Longue durée** : créé une fois, renouvelé jamais – TTL de plusieurs mois, aucune rotation
- **Visible** : tout mainteneur le lit dans les settings ; tout job peut l'exfiltrer (`echo $VAULT_TOKEN | base64` contourne le masquage)
- **Partagé** : tous les jobs, toutes les branches, tous les contributeurs ont la même identité – l'audit Vault ne voit qu'un seul acteur
- **Forks et MR** : selon la configuration, des pipelines de forks peuvent accéder aux variables – un contributeur externe aussi
- **Révocation aveugle** : en cas de fuite, on révoque /le token... et tous les pipelines tombent ensemble

Les autres anti-patterns du pipeline

Anti-pattern	Pourquoi c'est grave	À la place
Secrets copiés dans les variables de CI (<code>DB_PASSWORD</code> , <code>API_KEY</code> ...)	Deux sources de vérité : Vault tourne le secret, la CI garde l'ancien ; sprawl reconstitué	Le job lit Vault à chaque exécution
Token root ou quasi root « pour que ça marche »	Un pipeline compromis = tout Vault compromis	Une policy dédiée, moindre privilège
Policy wildcard (<code>path "secret/*"</code>) pour le pipeline	Le job de déploiement de PayLink lit les secrets de MediTrack	Une policy par service et par usage
Token de CI jamais révoqué en fin de job	Fenêtre d'exposition = TTL entier, pas la durée du job	TTL court + <code>vault token revoke -self</code>
Secrets affichés dans les logs de pipeline	Les logs sont lus par plus de monde que les settings, et archivés	Masquer, ou mieux : ne jamais afficher

Le principe : le pipeline est une app comme une autre

Toute la formation tient en trois règles — elles s'appliquent aussi au pipeline :

- **Jamais de secret en clair** dans le code, la config... ni les variables de CI
- **Une identité par service** — le pipeline de PayLink n'est ni celui de MediTrack, ni « la CI » en général
- **Le moindre privilège par défaut** — le job de déploiement lit `secret/paylink/api`, rien d'autre

“ Le pipeline est une application comme une autre : **il doit prouver son identité.** ”

La question devient : *comment un job de CI prouve-t-il qui il est, sans détenir de secret au départ ?*

- Module 2, AppRole : le « secret zéro » était livré par un tiers de confiance (wrapping)
- Module 5, Kubernetes : le ServiceAccount token était **émis et signé par la plateforme**
- Module 6, CI/CD : même idée — **la plateforme CI signe une preuve d'identité par job** → JWT/OIDC

2. Auth JWT/OIDC

La plateforme signe, Vault vérifie

Le principe : une preuve signée, aucun secret stocké

Trois acteurs, trois responsabilités :

- **La plateforme CI** (GitLab, GitHub...) détient une clé privée de signature. À chaque job, elle émet un **id_token** : un JWT qui atteste *ce job appartient au projet 42, branche main, pipeline 1337* – et le signe
- **Le job** reçoit cet id_token dans une variable, le présente à Vault :

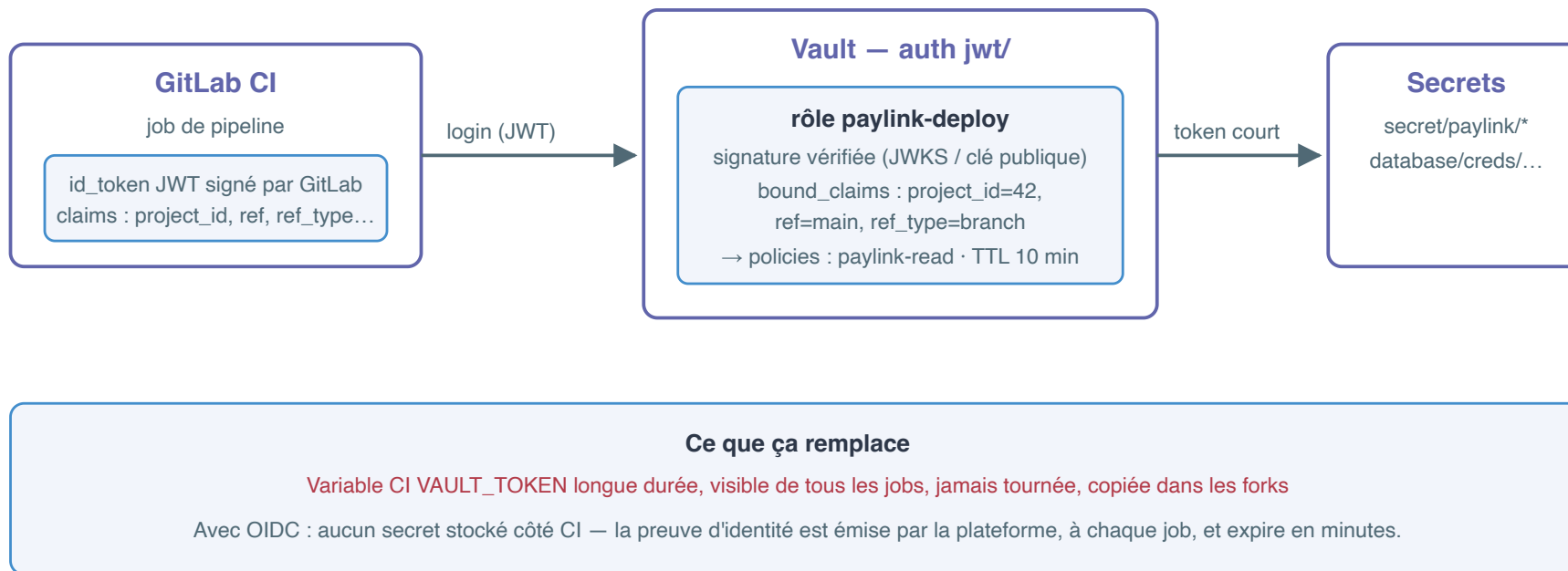
```
vault write auth/jwt/login role=... jwt=...
```

 Il ne détient aucun secret durable
- **Vault** vérifie la **signature** (via le JWKS de la plateforme – ses clés publiques – ou une clé publique déclarée), l'**issuer**, l'**audience**, puis compare les **claims** au contrat du rôle → token Vault à TTL court

Ce qui change tout :

- La preuve est **émise à chaque job**, expire en minutes, ne se stocke pas
- Elle est **infalsifiable** sans la clé privée – que seul l'émetteur possède
- Elle **décrit précisément** le contexte du job : projet, branche, protégée ou non...

Vue d'ensemble : ce que ça remplace



Anatomie d'un JWT : header.payload.signature

Un JWT : trois parties séparées par un point — rien n'est chiffré

le payload est lisible par tous ; la sécurité vient de la SIGNATURE, que seule la clé privée de la plateforme peut produire

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

. eyJpc3MiOiJodHRwczovL2dpdGxhYi5l...

. tK8mJp3lQw...

HEADER

base64url — l'algorithme

```
{ "alg": "RS256",  
  "typ": "JWT" }
```

comment la signature a été calculée

PAYLOAD — les claims

ce que la plateforme atteste sur CE job

```
{ "iss": "https://gitlab.example.com",  
  "aud": "https://vault.paylink.local",  
  "iat": "...", "nbf": "...", "exp": "...",  
  "project_id": "42",  
  "ref": "main",  
  "ref_protected": "true" }
```

valeurs GitLab = des CHAÎNES, pas des entiers

SIGNATURE

base64url

RS256(header.payload,
clé PRIVÉE GitLab)

modifier un claim invalide le JWT

Ce que Vault vérifie

1. la signature, via le JWKS (clés publiques)
2. l'issuer — bound_issuer
3. l'audience — bound_audiences
4. iat / nbf / exp — sensibles à l'horloge
5. les claims — bound_claims du rôle

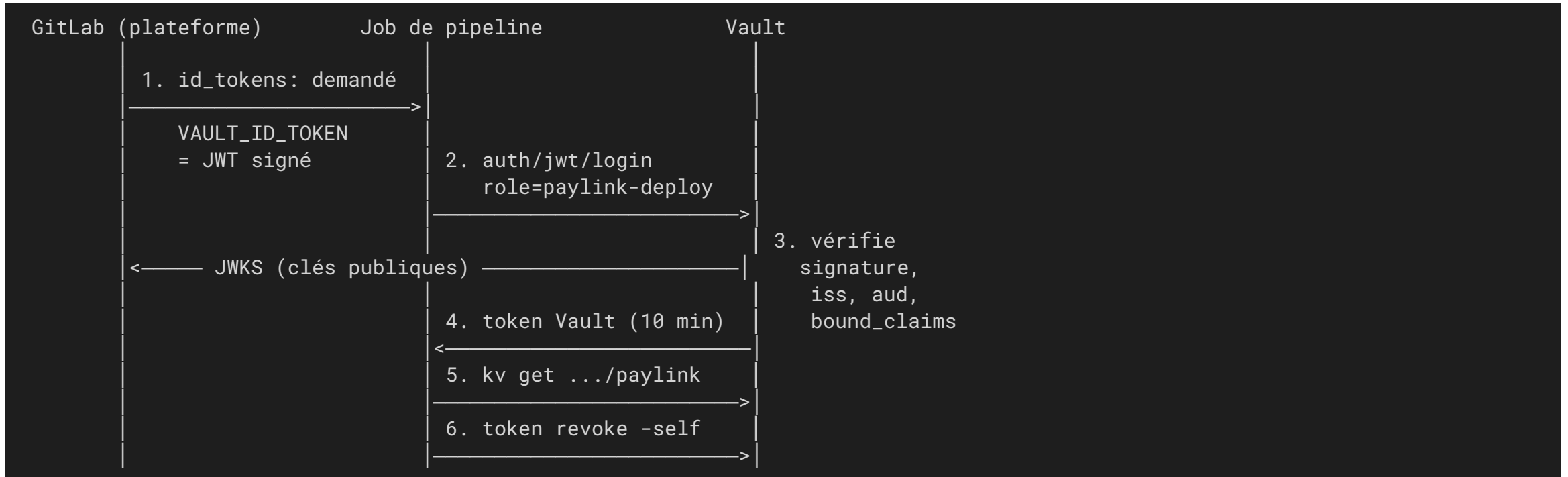
ref = main, project_id = 42 → login accepté

token Vault, TTL 10 min, policy paylink-read.
Le job n'a jamais détenu de secret durable.

ref = feature/hack → login REFUSÉ

claim "ref" does not match any associated bound claim values

Le flux complet : GitLab CI → Vault



Aucun secret stocké côté CI : la clé privée reste chez GitLab, Vault n'a que les clés publiques.

GitLab CI : `id_tokens:` et les claims du job

Le job demande explicitement un `id_token`, en fixant son **audience** :

```
deploy-paylink:
  id_tokens:
    VAULT_ID_TOKEN:
      aud: https://vault.paylink.local
```

GitLab remplit le payload avec les claims du job — ceux qu'on pourra exiger côté Vault :

Claim	Exemple	Usage côté Vault
<code>project_id</code> , <code>project_path</code>	"42", "utopios/paylink-api"	un rôle par projet
<code>ref</code> , <code>ref_type</code>	"main", "branch"	portée par branche (ou tag)
<code>ref_protected</code>	"true" / "false" (chaînes)	exiger une branche protégée
<code>pipeline_id</code> , <code>job_id</code>	"1337"	traçabilité dans l'audit
<code>iss</code> , <code>aud</code> , <code>sub</code>	issuer GitLab, audience demandée	vérifiés par la config et le rôle

Clés publiques exposées par l'instance : `https://gitlab.example.com/-/jwks`

GitHub Actions : OIDC natif

Même modèle, vocabulaire GitHub :

```
permissions:  
  id-token: write      # sans cette permission, aucun id_token n'est émis  
  contents: read
```

- L'issuer est commun à tout GitHub : `https://token.actions.githubusercontent.com`
- Claims utiles : `repository` (`"utopios/paylink-api"`), `ref` (`"refs/heads/main"`), `ref_protected`, `workflow`, `run_id`, `environment`
- L'audience se choisit à la demande du token (`jwtGithubAudience` avec `hashicorp/vault-action`)
- Côté Vault : `oidc_discovery_url="https://token.actions.githubusercontent.com"` — Vault découvre le JWKS tout seul

Attention au multi-tenant : l'issuer étant partagé par tout github.com, les `bound_claims` (`repository`, `ref`) sont **la seule** chose qui distingue votre dépôt d'un autre — soyez stricts.

Côté Vault : la configuration de la méthode

```
vault auth enable jwt

# Production, avec une vraie plateforme :
vault write auth/jwt/config \
  jwks_url="https://gitlab.example.com/-/jwks" \
  bound_issuer="https://gitlab.example.com"
```

- `jwks_url` (ou `oidc_discovery_url`) : Vault récupère et **rafraîchit** les clés publiques – les rotations de clés de la plateforme sont transparentes
- `bound_issuer` : tout JWT dont `iss` diffère est rejeté d'office
- Variante sans URL (notre lab, GitLab simulé) : déclarer la clé publique en dur

```
vault write auth/jwt/config \
  jwt_validation_pubkeys="$(cat gitlab-ci.pub)" \
  bound_issuer="https://gitlab.example.com"
```

Ne pas confondre : auth **jwt** = on présente un token déjà émis (machine à machine, CI/CD) ;
auth **oidc** = redirection navigateur pour un humain (SSO). Même plugin, deux modes.

Côté Vault : le rôle, un contrat d'identité

```
{
  "role_type": "jwt",
  "bound_audiences": "https://vault.paylink.local",
  "bound_claims": {
    "project_id": "42",
    "ref": "main",
    "ref_type": "branch"
  },
  "user_claim": "project_path",
  "token_policies": "paylink-read",
  "token_ttl": "10m",
  "token_max_ttl": "15m"
}
```

- `bound_audiences` : le claim `aud` doit correspondre — une audience **par usage**
- `bound_claims` : la portée — seule la branche `main` du projet 42 passe ; valeurs en **chaînes** (`"42"`)
- `user_claim` : le claim qui nomme l'entité côté Vault (audit, aliases)
- `token_ttl` court : un job vit quelques minutes, son token aussi
- Chaque clause refuse **indépendamment** — et l'erreur de Vault nomme le claim fautif

Piège réel n° 1 : `bound_claims` est une map

Tentation naturelle avec la CLI — et sortie réelle du lab :

```
v write auth/jwt/role/paylink-deploy bound_claims="project_id=42,ref=main" ...
```

```
Error writing data to auth/jwt/role/paylink-deploy: Error making API request.
```

```
URL: PUT http://127.0.0.1:8200/v1/auth/jwt/role/paylink-deploy
```

```
Code: 400. Errors:
```

```
* error converting input for field "bound_claims": '' expected a map, got 'string'
```

Correction : passer le corps **en JSON** via l'entrée standard —

```
vault write auth/jwt/role/paylink-deploy - <<EOF  
{ "bound_claims": { "project_id": "42", "ref": "main" }, ... }  
EOF
```

Effet domino à connaître : après cet échec, le login renvoie

`role "paylink-deploy" could not be found` — l'erreur visible n'est pas la cause racine.

Piège réel n° 2 : la dérive d'horloge (**nbf**)

Sortie réelle rencontrée en préparant ce lab :

```
Code: 400. Errors:
```

```
* error validating token: invalid not before (nbf) claim: token not yet valid
```

Cause : l'horloge de la **VM podman** dérive (mise en veille du portable...). Le JWT, généré avec l'heure de l'hôte, a un **nbf** « dans le futur » pour Vault.

Diagnostic : comparer `date -u +%s` (hôte) et `podman exec vault date -u +%s` (conteneur).

Corriger la dérive d'horloge

Resynchroniser l'horloge de la machine podman :

```
podman machine ssh sudo hwclock -s  
# ou : podman machine ssh sudo systemctl restart systemd-timesyncd  
# ou : podman machine stop && podman machine start
```

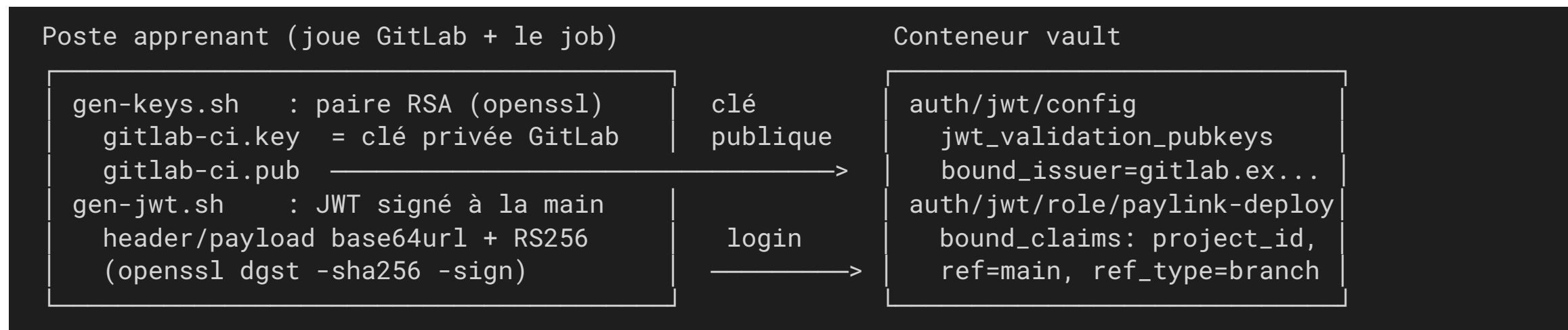
Parade retenue dans le lab — `gen-jwt.sh` prend l'heure **du conteneur** pour `iat` / `nbf` / `exp` :

```
NOW=$(podman exec vault date -u +%s)
```

En production le symptôme existe aussi : surveillez la **synchronisation NTP** de vos nœuds Vault.

Notre lab : GitLab est simulé

Pas de GitLab dans le lab — on fabrique nous-mêmes ce que GitLab fabrique, et on voit tout :



- Rôle `paylink-deploy` : `bound_claims project_id=42, ref=main, ref_type=branch` ; `user_claim=project_path` ; policy `paylink-read` ; TTL 10 min
- Le login `main` réussit ; le login `feature/hack` est refusé
- Intérêt pédagogique : un JWT n'a rien de magique — 30 lignes de shell et `openssl` suffisent à en produire un ; seule la **clé privée** compte



Démo 6.1

Auth JWT : le pipeline prouve son identité

`demos/module-06/demo-6-1-jwt-gitlab.md` — clés, config, rôle, login accepté/refusé
— 20 min

Le contrat en action (sorties réelles du lab)

Login avec le JWT de la branche `main` — accepté :

```
token_duration      10m
token_policies      ["default" "paylink-read"]
token_meta_role      paylink-deploy
```

Login avec le JWT de `feature/hack` — **signé et cryptographiquement valide**, mais :

```
Code: 400. Errors:
```

```
* error validating claims: claim "ref" does not match any associated bound claim values
```

À retenir :

- Le refus ne vient pas de la signature : le contenu attesté viole le contrat du rôle
- Vault nomme le claim fautif — lire le message d'erreur est le premier réflexe de dépannage
- TTL 10 min, une seule policy applicative : moindre privilège, courte durée



Exercice 6.1

Durcir le rôle JWT du pipeline PayLink

`exercices/module-06/exercice-6-1-bound-claims.md` — branche protégée, audience dédiée, TTL réduits — 20 min

3. Pipelines réels

**GitLab CI et GitHub Actions — fournis en référence,
non exécutés en lab**

`.gitlab-ci.yml` — le job se logue lui-même (CLI)

Fichier complet : `code/module-06/exemple.gitlab-ci.yml` — référence, non exécuté en lab (le lab simule GitLab).

```
deploy-paylink:
  stage: deploy
  image: docker.io/hashicorp/vault:1.20
  rules:
    - if: $CI_COMMIT_REF_NAME == "main"          # cohérent avec bound_claims ref=main
  id_tokens:
    VAULT_ID_TOKEN:
      aud: https://vault.paylink.local           # = bound_audiences du rôle
  variables:
    VAULT_ADDR: https://vault.paylink.example.com
  script:
    - export VAULT_TOKEN="$(vault write -field=token auth/jwt/login role=paylink-deploy jwt=$VAULT_ID_TOKEN)"
    - export PAYLINK_API_KEY="$(vault kv get -field=api_key secret/paylink/api)"
    - echo "Déploiement de paylink-api"          # ... étapes qui consomment le secret ...
    - vault token revoke -self                    # fin de job : rien ne survit
```

Les 5 phases du TP 6 s'y retrouvent : id_token → login → lecture → usage → révocation.

`.gitlab-ci.yml` – l'intégration `secrets:` native

GitLab Premium/Ultimate fait le login et l'injection à votre place – **référence, non exécuté en lab** :

```
deploy-paylink-native:
  id_tokens:
    VAULT_ID_TOKEN:
      aud: https://vault.paylink.local
  variables:
    VAULT_SERVER_URL: https://vault.paylink.example.com
    VAULT_AUTH_PATH: jwt
    VAULT_AUTH_ROLE: paylink-deploy
  secrets:
    PAYLINK_API_KEY:
      vault: paylink/api/api_key@secret      # <path>/<field>@<mount kv-v2>
      token: $VAULT_ID_TOKEN
      file: false                            # variable d'env plutôt que fichier
  script:
    - echo "PAYLINK_API_KEY injectée (masquée dans les logs)"
```

L'intégration native : ce qu'il faut savoir

- **Moins de code** dans le `script:`, masquage automatique des valeurs
- **Piège classique** : par défaut `file: true` — GitLab écrit le secret dans un fichier temporaire et met **son chemin** dans la variable, pas la valeur. D'où le `file: false` ci-contre.
- **Côté Vault** : strictement la même configuration (méthode `jwt`, rôle, `bound_claims`). L'intégration native ne change que le *client*.

GitHub Actions (1/2) – déclencheur et permissions

Fichier complet : `code/module-06/exemple-github-actions.yml` – référence, non exécuté en lab.

```
name: deploy-paylink
on:
  push:
    branches: [main]
permissions:
  id-token: write          # indispensable : autorise l'émission de l'id_token
  contents: read
```

Sans `id-token: write`, **aucun id_token n'est émis** et le login échoue :
c'est l'oubli n° 1 sur GitHub Actions.

GitHub Actions (2/2) – l'étape Vault

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/vault-action@v3
        with:
          url: https://vault.paylink.example.com
          path: jwt-github
          method: jwt
          role: paylink-deploy-gh
          jwtGithubAudience: https://github.com/utopios
          secrets: |
            secret/data/paylink/api api_key | PAYLINK_API_KEY
```

Côté Vault : rôle avec `bound_claims repository=utopios/paylink-api, ref=refs/heads/main` – seule barrière dans un issuer partagé par tout github.com.

4. Choisir son intégration

Synthèse des patterns vus pendant la formation

Patterns d'intégration – hors Kubernetes

Pattern	Vu au	Comment	Quand le choisir
SDK / API directe	M1-M2	L'app appelle l'API Vault (AppRole...), gère renouvellements	App dont on maîtrise le code, besoin de secrets dynamiques finement gérés
Vault Agent sidecar	M5	Processus à côté de l'app : auto-auth, cache, templates, re-rendu	App non modifiable, secrets rotatifs, hors ou dans K8s
CI/CD JWT-OIDC	M6	La plateforme signe un id_token par job, Vault vérifie les claims	Pipelines : build, deploy, migrations – jamais de token stocké

Patterns d'intégration – dans Kubernetes

Pattern	Vu au	Comment	Quand le choisir
Agent init (pre-populate)	M5	Rendu des secrets une fois avant le démarrage de l'app	Secrets statiques lus au boot, pas de rotation à chaud
Agent Injector	M5	Annotations sur le pod → sidecar/init injectés automatiquement	K8s, adoption rapide sans toucher aux images ni aux charts
VSO (operator)	M5	CRD → Secrets K8s natifs synchronisés, rollout restart	K8s, charts tiers qui exigent des Secrets natifs, GitOps

Règle commune, quel que soit le pattern : **une identité par consommateur, une policy au moindre privilège, des TTL courts.**

Comment choisir : les questions à se poser

1. Qui consomme ?

- Un pipeline → JWT/OIDC, portée par projet et par branche
- Une app dans Kubernetes → auth kubernetes + Injector ou VSO
- Une app hors K8s → AppRole (+ Vault Agent si le code ne doit pas changer)

2. Peut-on modifier le code ?

- Oui → SDK/API directe : contrôle total (renew, retry, secrets dynamiques)
- Non → Agent (sidecar/init), Injector, VSO : les secrets arrivent en fichiers ou Secrets K8s

3. Le secret tourne-t-il pendant la vie du processus ?

- Oui → sidecar (re-rendu) ou VSO (+ rollout restart) ; attention au re-rendu ~5 min des statiques
- Non → init container ou lecture au démarrage suffisent

4. Le consommateur exige-t-il un Secret K8s natif ? (chart tiers, `envFrom`)

- Oui → VSO ; sinon préférer les fichiers injectés (le Secret K8s est une copie de plus)

En cas de doute : le pattern le plus simple qui respecte « zéro secret stocké » gagne.

Check-list — identité et droits

- [] **Une identité par service** et par pipeline — jamais de token partagé, jamais de root
- [] **Moindre privilège** : une policy par usage, pas de wildcard ; revue des policies comme du code
- [] Portée CI/CD par **bound_claims** : projet + branche (+ `ref_protected`), audience dédiée

Check-list — cycle de vie

- [] **TTL courts** partout : token de job ≤ 10 min, secrets dynamiques ajustés au besoin réel
- [] **Révocation** : `token revoke -self` en fin de job ; procédure de révocation d'urgence connue (token, lease, secret_id)
- [] **Rotation** : privilégier les secrets dynamiques (module 4) aux statiques copiés

Check-list — hygiène

- [] **Jamais de secret dans les logs** (CI, applicatifs) ni dans les artefacts committés
- [] **Audit activé** et exploité : qui a lu quoi, avec quelle identité, quand
- [] **En cas d'incident** : révoquer d'abord, tourner le secret ensuite, analyser l'audit enfin

 TP 6

Le pipeline PayLink sans aucun secret stocké

`tp/module-06/tp-6-pipeline-jwt.md` — job simulé complet : id_token, login, lecture, artefact, révocation — 40 min

Récap – Jour 1 : consommer Vault

- **M1 – Concepts, CLI & API**

mounts, paths, tokens, leases ; KV v2 (`data/`, versions) ; l'API REST et `X-Vault-Token` – les premiers secrets de PayLink

- **M2 – Authentification applicative**

jamais de token statique ; AppRole (`role_id` + `secret_id`), le « secret zéro » résolu par le response wrapping ; principes K8s et cloud

- **M3 – Politiques et moindre privilège**

HCL, déni par défaut, pièges KV v2 ; politiques **templâtées** avec entités/aliases – PayLink et MediTrack cloisonnés

Récap – Jour 2 : intégrer Vault

- **M4 – Secrets dynamiques & transit**

comptes PostgreSQL éphémères, leases (renew/revoke) ; chiffrement as a service, rotation de clés, rewrap – les cartes de PayLink

- **M5 – Kubernetes**

Vault Agent (auto-auth, templates), Injector par annotations, VSO (CRD → Secrets natifs), CSI – MediTrack sur kind

- **M6 – CI/CD & bonnes pratiques**

JWT/OIDC, bound_claims, le pipeline sans secret stocké

Le fil conducteur, du premier au dernier module : **jamais de secret en clair ; une identité par service ; le moindre privilège par défaut.**

Quiz – questions 1 et 2

Question 1 — Pourquoi une variable de CI `VAULT_TOKEN` est-elle un anti-pattern, même « Protected » et « Masked » ?

- A. Parce que Vault refuse les connexions venant des runners CI
- B. Parce qu'elle est longue durée, lisible par les mainteneurs, partagée par tous les jobs et jamais tournée
- C. Parce que les variables GitLab sont limitées à 255 caractères
- D. Parce que le masquage des logs ralentit le pipeline

Question 2 — Dans le flux OIDC CI/CD, qui signe l'id_token présenté à Vault ?

- A. Le job de pipeline, avec un secret de projet
- B. Vault, lors du premier login
- C. La plateforme CI (GitLab, GitHub), avec sa clé privée
- D. Le développeur, avec sa clé SSH

Quiz – questions 3 et 4

Question 3 – À quoi sert le JWKS (`/-/jwks` chez GitLab) dans ce flux ?

- A. À fournir à Vault les clés publiques pour vérifier la signature des id_tokens
- B. À stocker les secrets du projet côté GitLab
- C. À chiffrer le payload du JWT
- D. À générer les tokens Vault côté plateforme

Question 4 – Vous créez un rôle jwt et la CLI répond

`error converting input for field "bound_claims": '' expected a map, got 'string'`. Que corrigez-vous ?

- A. Le TTL, qui doit être exprimé en secondes
- B. Le nom du rôle, qui contient un tiret
- C. La policy, qui n'existe pas encore
- D. La syntaxe : `bound_claims` doit être passé en JSON (map), par exemple via un heredoc sur l'entrée standard

Quiz – questions 5 et 6

Question 5 – Un login JWT échoue avec `invalid not before (nbf) claim: token not yet valid` sur votre lab podman. Quelle est la cause la plus probable ?

- A. Le rôle n'existe pas
- B. L'horloge de la VM podman a dérivé par rapport à celle qui a généré le JWT
- C. La policy `paylink-read` a été supprimée
- D. Le JWT a été signé avec le mauvais algorithme

Question 6 – Dans un `.gitlab-ci.yml`, comment un job demande-t-il un `id_token` pour Vault ?

- A. Avec le mot-clé `id_tokens:` et une audience `aud:`
- B. En définissant la variable `VAULT_TOKEN` dans les settings
- C. Avec `permissions: id-token: write`
- D. En appelant `gitlab-runner token create` dans le script

Quiz – questions 7 et 8

Question 7 – Dans GitHub Actions, que faut-il déclarer pour que le workflow puisse obtenir un id_token OIDC ?

- A. `secrets: OIDC_TOKEN` dans le dépôt
- B. `permissions: id-token: write` dans le workflow
- C. Une variable d'environnement `GITHUB_OIDC=true`
- D. Un runner auto-hébergé

Question 8 – Un JWT parfaitement signé, émis pour `ref=feature/hack`, est présenté au rôle `paylink-deploy` (`bound_claims` `ref=main`). Que se passe-t-il ?

- A. Login accepté avec des droits réduits
- B. Login accepté : la signature est valide
- C. Login refusé : le claim `ref` ne correspond à aucune valeur attendue par le rôle
- D. Login refusé : les JWT de branches ne sont jamais acceptés

Quiz – questions 9 et 10

Question 9 – Une équipe déploie sur Kubernetes un chart Helm tiers qui exige un Secret K8s natif consommé en `envFrom`, avec propagation de la rotation. Quel pattern de la formation choisir ?

- A. AppRole avec livraison wrappée
- B. Vault Agent Injector avec sidecar
- C. Auth JWT depuis le pipeline
- D. Vault Secrets Operator avec `VaultStaticSecret` et rollout restart

Question 10 – Que doit faire le job de CI de son token Vault en fin d'exécution ?

- A. L'exporter en artefact pour le job suivant
- B. Le stocker dans les variables du projet pour le prochain pipeline
- C. Le révoquer (`vault token revoke -self`) – son TTL court fait le reste en cas d'oubli
- D. Le transmettre à l'application déployée

Clôture — votre plan d'action

Trois questions à vous poser en rentrant

1. **Où dorment vos secrets aujourd'hui ?** — variables de CI, fichiers `.env`, wikis : lesquels migrer vers Vault en premier ?
2. **Qui est chaque consommateur ?** — quelles apps et pipelines partagent une identité ou un token, et quel pattern (AppRole, K8s, JWT) leur donner ?
3. **Que se passe-t-il si ça fuit ce soir ?** — savez-vous révoquer, tourner, auditer — et en combien de temps ?

Merci pour ces deux jours — la cheatsheet de chaque module vous accompagne : `cheatsheet/`